

Table of Contents

Table of Contents



ROOF-AID CRM

Complete Developer & Business Specification

Version 4.0 | March 2026 | CONFIDENTIAL

Multi-Tenant SaaS CRM Platform for the Roofing Industry

#	Section
1	Executive Summary & Product Vision
2	Business Model, Revenue & Monetization
3	SaaS Tiers & Feature Flags
4	Platform Compatibility & Operating Systems
5	Cloud Architecture, Infrastructure & Code Standards
6	User Roles & Permissions
7	Core CRM — Prospect Pipeline & Dashboard
8	Communication Modules (Phone / SMS / Email)
9	Documents & Contracts Module
10	Appointments & Scheduling
11	Admin, Analytics & Reporting
12	Mobile Application (iOS & Android)
13	Storm Leads Module (Tier 2)
14	AI Calling Agent (Tier 3)
15	Insurance Supplements & Commission Billing (Tier 4)
16	AI Roof Inspection — Computer Vision (Tier 5)
17	Multi-Tenant Architecture
18	Billing, Subscriptions & Payment Processing
19	Website, Marketing & Acquisition Integrations
20	Customer Onboarding & Self-Service
21	Customer Service & AI Support Agent
22	Security, Compliance & App Store Requirements
23	Notifications, Webhooks & Integrations
24	Business Processes & Operational Workflows
25	Edge Cases & Scenarios Not to Miss

26	Development Phases & Roadmap
27	Scaling Strategy — Firebase to Cloud-Agnostic Architecture
28	Glossary
29	Proprietary Modules & IP Licensing
30	Anti-Collision System (Sistema Anti-Colision)
31	Existing Production System & v1 to v2 Migration
1.	Executive Summary & Product Vision

Roof-Aid CRM is a vertical SaaS platform built exclusively for the roofing industry. It unifies the entire business workflow — from storm-triggered lead generation, automated or human calling, appointment scheduling, field inspection documentation, contract signing, insurance supplement filing, and AI-powered damage assessment — into one cloud-based system accessible from any device, anywhere in the world.

The platform is sold as a tiered monthly subscription to roofing companies. Each tier unlocks more automation and intelligence. The business model is designed so that as customers grow, they naturally upgrade — creating a powerful upsell engine with recurring revenue at every level.

This document is the complete specification for the developer(s) who will build this system from scratch. Every feature, screen, data model, integration, workflow, security requirement, and business process is described here. Nothing should need to be assumed.

1.1 Core Design Principles

- Cloud-native — 100% cloud hosted, zero on-premise, no installation required by any user
- Multi-tenant — one codebase, one infrastructure serves all customers with complete data isolation
- Mobile-first — the web app is fully responsive; a native iOS/Android app serves field inspectors
- Modular & tiered — every feature is a module; subscription tier controls access via feature flags
- Lean & efficient — minimal dependencies, no bloat, every line of code justified
- Secure by design — security is built into every layer, not added at the end
- App-store ready — mobile app meets Apple App Store and Google Play Store requirements from day one
- Integration-ready — clean API boundaries for AI, ML, and third-party services; stubs exist for all future modules
- Revenue-generating at every layer — every tier adds measurable value and justifies the price increase

2. Business Model, Revenue & Monetization

Roof-Aid generates revenue through five distinct streams. Each stream corresponds to a product tier. Understanding the business model is critical for the developer because it directly drives architectural decisions — the commission tracking system, billing integration, and feature flag architecture must all reflect these revenue models.

2.1 Revenue Streams

Stream	Model	Description
Tier 1 — CRM	Monthly subscription	Roofing companies pay a recurring monthly fee for the core CRM. Priced per tenant or per seat.
Tier 2 — Leads	Higher monthly subscription	Adds storm lead data feed. Higher recurring monthly fee.
Tier 3 — Calls	Higher monthly subscription	Adds AI calling agent on top of human softphone. Premium recurring monthly fee.
Tier 4 — Supplements	10% commission per transaction	No subscription increase — Roof-Aid takes 10% of each supplement claim value processed through the platform.
Tier 5 — AI Inspection	Per-inspection fee or included in top tier	Fee per AI damage assessment report generated, OR bundled into a premium tier subscription.

2.2 Supplement Commission Model — Technical Requirements

This is the most unique and complex revenue stream. Every supplement transaction must be tracked, billed, and auditable.

- Each supplement document has a `claimValue` field (USD amount) entered by the user or auto-estimated
- When a supplement is submitted, the system automatically calculates commission: $\text{claimValue} \times 0.10$
- A `commission_transactions` Firestore collection records: `tenantId`, `supplementId`, `claimValue`, `commissionAmount`, `status` (pending/billed/paid/disputed), `createdAt`
- Stripe is used to charge the tenant for the commission — either via automatic monthly invoice or immediate charge
- Tenants see a Supplements & Billing dashboard showing all supplements, claim values, and commissions owed
- Super Admin sees platform-wide commission revenue: total processed, collected, pending
- Monthly commission statement PDF auto-generated and emailed to each tenant at month end

- Dispute workflow: tenant can flag a supplement commission for review; status changes to disputed; Super Admin resolves
- If a supplement claim is later denied by insurance, tenant can request commission credit (manual review)

2.3 Pricing Architecture

- All prices are configurable by Super Admin — never hardcoded in the application
- Prices stored in a platform_config Firestore document accessible only by Super Admin
- Trial period: 14 days free on Tier 1 for all new tenants — trial_expires_at stored in tenant record
- Grace period: 7 days after payment failure before account suspended — data preserved
- Downgrade: features disabled immediately but all data preserved — re-enable instantly on upgrade
- Annual billing option: discounted rate vs monthly — stored as billing_cycle in tenant record
- Per-seat pricing option: priced per active user per month instead of flat tenant rate — configurable per plan

2.4 White-Label & Reseller Potential

- Platform must support white-labeling in the future — company name, logo, colors are tenant-level config
- Email sending domain is per-tenant (SendGrid subusers) — tenants appear to use their own email
- URL structure: {company}.roofaid.app — future support for fully custom domains
- Reseller tier: a third party (e.g. roofing materials distributor) can sell Roof-Aid to their contractor customers

3. SaaS Tiers & Feature Flags

Tiers are implemented as feature flags stored in the tenant's Firestore document. The same codebase serves all tiers — access is controlled programmatically. This means upgrades and downgrades take effect instantly without any re-deployment or re-provisioning.

3.1 Tier Definitions

Tier	Name	Monthly Price*	Included Features
1	CRM Core	TBD	Prospect management, map search, human calling (softphone), SMS, email, appointments, document

			generation, e-signature, admin panel, mobile app (Rufero)
2	CRM + Leads	TBD	Everything in Tier 1 + hail storm lead auto-import, storm radar map overlay, lead source tracking
3	CRM + Leads + AI Calls	TBD	Everything in Tier 2 + AI calling agent (auto-dials, qualifies, schedules), call transcripts, AI call analytics
4	Supplements	Commission-based	Everything in Tier 3 + ML supplement writing assistant + 10% commission billing
5	AI Inspection	TBD per inspection	Everything in Tier 4 + computer vision roof damage analysis from photos

- Prices set by Super Admin in platform_config — not hardcoded.

3.2 Feature Flag Schema

Flag	Type	Controls
features.crmCore	Boolean (always true)	Base CRM — always enabled
features.leads	Boolean	Storm lead import, radar overlay
features.aiCaller	Boolean	AI outbound calling agent
features.humanCalling	Boolean	Telnyx WebRTC softphone (enabled Tier 1+)
features.supplements	Boolean	ML supplement module
features.supplementCommission	Boolean	Auto-commission billing (10% via Stripe). SEPARATE from features.supplements — writing tool can be enabled without auto-billing, supporting manual billing arrangements during early sales
features.computerVision	Boolean	AI roof inspection (future)
features.mobileApp	Boolean	Rufero mobile app access
features.advancedAnalytics	Boolean	Extended reporting and BI dashboard
features.apiAccess	Boolean	REST API access for external integrations
features.whiteLabel	Boolean	Custom branding (future)

4. Platform Compatibility & Operating Systems

The platform must run on every major device and operating system. Users include office staff on Windows PCs, field inspectors on Android phones and iPads, and business owners on MacBooks. The system must never require a specific device, OS, or browser plugin.

4.1 Web Application — Browser & OS Support

Platform	Browser	Support	Notes
Windows 10/11	Chrome 90+	Full	Primary office environment
Windows 10/11	Edge 90+	Full	Common in corporate environments
Windows 10/11	Firefox 90+	Full	
macOS 12+	Chrome	Full	
macOS 12+	Safari 14+	Full	WebRTC must be explicitly tested — Safari has quirks
macOS 12+	Firefox	Full	
iPadOS 15+	Safari	Full	Field inspectors may use iPads
iPadOS 15+	Chrome	Full	
Android 10+	Chrome	Full	Telefonistas on Android phones/tablets
iOS 15+	Safari	Full	Only full browser on iOS
ChromeOS	Chrome	Full	Some office users on Chromebooks
Linux	Chrome/Firefox	Best Effort	Not primary target but must not break

4.2 Mobile App — Native App Support

Platform	Min Version	Target Devices	Distribution
iOS	iOS 15+	iPhone 11+, all iPad models	Apple App Store
Android	Android 10 (API 29)+	All major Android phones and tablets	Google Play Store

4.3 Progressive Web App (PWA)

- Required Phase 1 deliverable. The web app must be installable as a PWA on any device — adds to home screen, works offline
- Service Worker required — caches critical assets for offline use
- Web App Manifest required — name, icons, theme colors, display mode
- PWA provides desktop-like experience on Windows/Mac without a native app download
- Offline queue: actions taken offline (status updates, notes) sync when connection restores

- Phase 1 required files: manifest.json (icons, theme colors, display mode), service-worker.js (cache strategy + background sync), offline fallback page, and queued write sync. Must be tested on Chrome, Safari iOS, and Android Chrome

4.4 No Desktop Native App

No native Windows, macOS, or Linux desktop app is required. The PWA covers all desktop use cases. This reduces build and maintenance cost significantly.

4.5 Responsive Design Requirements

- Minimum supported width: 375px (iPhone SE)
- Fully responsive from 375px to 2560px — no horizontal scrolling on any screen
- Touch-optimized: minimum tap target size 44x44px (Apple HIG standard)
- All modals, dropdowns, and forms must work on touch screens without a mouse
- Map interactions (tap, pinch-zoom, right-click equivalent) must work on touch

5. Cloud Architecture, Infrastructure & Code Standards

The platform is 100% cloud-hosted on Google Firebase and associated Google Cloud services. There is no server to manage, no infrastructure to provision, and no on-premise component. The architecture must be serverless, auto-scaling, and cost-efficient.

5.1 Infrastructure Stack

Layer	Service	Purpose
Frontend Hosting	Firebase Hosting	Global CDN, automatic SSL/TLS, custom subdomains
Backend Compute	Cloud Functions Gen2 (Node.js 22)	Serverless — scales to zero, no idle cost
Database	Cloud Firestore (Native Mode)	Real-time NoSQL, globally distributed, offline-capable
File Storage	Firebase Cloud Storage	Documents, photos, audio recordings
Authentication	Firebase Auth	Accounts, JWT tokens, custom role claims
Telephony	Telnyx	WebRTC calling, SMS, AI call control, personal extensions
Email	SendGrid	Transactional email, per-user subaccounts, inbound parsing
Payments	Stripe	Subscription billing, commission invoicing, webhooks
Push Notifications	Firebase Cloud Messaging (FCM)	Mobile push, web push via Service Worker

Monitoring	Firebase Performance + GCP Cloud Logging	Errors, latency, usage, cost alerts
Secrets	Google Secret Manager	API keys, credentials — never in code or env files
CI/CD	GitHub Actions	Automated deploy to staging and production on merge

5.2 Code Quality Standards — Non-Negotiable

These are requirements, not suggestions. Every pull request must meet these standards before merging.

Architecture:

- Single Responsibility Principle — every function does exactly one thing
- DRY (Don't Repeat Yourself) — shared logic extracted into utility modules
- No dead code — unused functions, variables, imports removed before merge
- No hardcoded values — all config in environment variables, Secret Manager, or Firestore config
- Feature flags checked at runtime — never compile-time conditionals for tier gating

Backend (Cloud Functions):

- Every function validates inputs before any DB operation — reject invalid requests immediately
- Every function has structured error handling — errors logged with context, never silently swallowed
- All secrets from Google Secret Manager — never process.env for sensitive values
- Firestore connection pooled — do not create new connections per function invocation
- No collection scans — every query uses indexed fields with tenantId as first composite field
- Batch writes for any operation writing more than 1 document
- Transactions for any operation requiring atomicity
- Max instances set per function to prevent runaway costs
- Min instances = 1 for latency-critical functions (auth, calling, real-time updates)

Frontend:

- Bundle size target: under 650KB gzipped total (accounts for React ~45KB + state management ~15KB on top of the measured 580KB production baseline from the existing system)
- Lazy-load all non-critical modules — route-based code splitting
- No blocking scripts in critical render path
- All user-generated content sanitized with DOMPurify before rendering
- Zero console.log in production — structured logging only

5.2.1 Firebase v10+ Compatibility — REQUIRED

Firebase v10+ (released May 2025) introduced breaking changes. All development must target Firebase SDK v10+ from the first line of code. Do NOT use v9 compat layer.

- Modular imports only: Continue using `import { getFirestore } from 'firebase/firestore'` pattern throughout
- Auth persistence: Use `browserLocalPersistence` / `inMemoryPersistence` constants (v10 API). Test on Safari iOS which has additional restrictions on persistence
- Firestore listeners: `onSnapshot()` return value handling changed. Test all real-time listeners against v10 emulator
- Storage URLs: Signed URL format changed in v10. All `getDownloadURL()` calls must be tested against v10 behavior
- Required: Run full Firebase Emulator Suite (v10) before any staging or production deployment

5.3 Environments

Environment	Firebase Project	Purpose	Data
Local Development	Firebase Emulator Suite	Developer machines — no real services hit	Synthetic test data only
Staging	Separate Firebase project	Pre-production QA and integration testing	Anonymized test data
Production	Separate Firebase project	Live customer data	Real customer data — restricted access

5.4 Performance Targets

Metric	Target
First Contentful Paint (4G)	< 1.5 seconds
Time to Interactive (4G)	< 3.0 seconds
Dashboard load (50 prospects)	< 500ms

Real-time update latency	< 400ms
Firestore query (indexed)	< 50ms average
PDF generation	< 5 seconds
Cloud Function cold start	< 3 seconds (p95)
Cloud Function warm execution	< 500ms (p95)
Mobile app launch to usable	< 2.5 seconds
Lighthouse score	90+ on all metrics

5.5 Disaster Recovery

- Firestore daily automatic backups — 30-day retention
- Cloud Storage versioning enabled on all tenant buckets
- Recovery Time Objective (RTO): < 4 hours
- Recovery Point Objective (RPO): < 24 hours
- Zero-downtime deployments — new Cloud Functions deploy alongside existing; traffic switches atomically

6. User Roles & Permissions

Six roles exist in the system. Five are operational roles, one is platform-level. Roles are stored as custom claims in Firebase Auth JWT tokens and enforced in Firestore Security Rules — frontend role checks are UX only, not security.

Role	Scope	Description
Super Admin	Platform	Roof-Aid platform owner. Manages all tenants, subscriptions, billing, and platform config. Invisible to tenants.
Owner	Tenant	Roofing company owner. Full tenant access including billing. One per tenant.
Admin	Tenant	Office manager. Full prospect and user access. Cannot access billing. Multiple per tenant.
Telefonista	Tenant	Call agent. Can search, contact, schedule, and update all prospects. No admin access.
Rufero	Tenant	Field inspector. Sees only assigned prospects. Uses mobile app. Uploads photos and signatures.
AI Agent	System	Automated system role for AI caller actions — logged with source=ai-agent

6.1 Permissions Matrix

Capability	Super Admin	Owner	Admin	Telefonista	Rufero
------------	-------------	-------	-------	-------------	--------

View all prospects	YES	YES	YES	YES	Assigned only
Create/edit prospect	YES	YES	YES	YES	Limited
Delete prospect	YES	YES	YES	NO	NO
Import prospects (CSV)	YES	YES	YES	NO	NO
Assign prospect to Rufero	YES	YES	YES	NO	NO
Make call (softphone)	NO	YES	YES	YES	NO
AI agent trigger	NO	YES	YES	NO	NO
Send SMS	NO	YES	YES	YES	YES (mobile)
Send Email	NO	YES	YES	YES	NO
Schedule appointment	NO	YES	YES	YES	View/update only
Generate document	NO	YES	YES	YES	YES (mobile)
Capture e-signature	NO	NO	NO	NO	YES (mobile)
Upload inspection photos	NO	NO	NO	NO	YES (mobile)
Create supplement	NO	YES	YES	YES	NO
View activity logs	YES	Tenant	Tenant	Own only	NO
Manage users	YES	YES	NO	NO	NO
Manage tenant settings	YES	YES	NO	NO	NO
Access analytics	Platform	Tenant	Tenant	Own stats	NO
Manage billing	YES	YES	NO	NO	NO
Access Super Admin panel	YES	NO	NO	NO	NO

7. Core CRM — Prospect Pipeline & Dashboard

7.1 Pipeline Statuses

Status	Icon	Meaning	Who Triggers
New Leads		Just imported or entered — no contact made	System (import) or Admin
Prospects		Reviewed and queued for outreach	Telefonista or Admin
Contacted		Successful contact made	Telefonista after call/SMS
Scheduled		Inspection booked and	System when appointment

(Appointment)		confirmed	created
Closed Customer	✓	Contract signed — job won	Admin after document signed
Not Viable	✗	Not interested, unreachable, or disqualified	Telefonista or Admin

7.2 Dashboard — Main Screen

Filter & Search Bar:

- City dropdown — populated from tenant’s city list
- Status dropdown — 6 pipeline statuses
- Quick Filter text box — real-time client-side search by name, address, or phone
- Query Database button — executes Firestore query
- Radar toggle — storm overlay on map (Tier 2+ only; hidden if not enabled)

Results List (Left Panel):

- Loads 60 records per page (required by the Anti-Collision System in Section 30, which rotates display order across a 0-59 second cycle). “Load 60 More” button for next page
- Result count: ‘50 de 150’ format
- Each card: name, address, home value, hail size, city, tipo, and 6 action buttons
- Card buttons: Call | SMS | Email | Appt | Go | Notes
- DNC is NOT on the card. It is accessible from the full prospect profile only to prevent accidental flagging
- Clicking a map pin highlights the corresponding card and scrolls to it

Map (Right Panel):

- Google Maps with all results as color-coded pins by status
- Right-click on any map point opens Proximity Search modal: center point auto-set from click position, radius selector (5/10/25/50 miles), status filter, Search button. The topbar proximity button uses current GPS location as center
- Map auto-zooms to fit all current results
- Storm radar overlay layer (Tier 2+)

7.3 Prospect Full Profile

Tab/Section	Content
-------------	---------

Overview	Name, address, city, state, ZIP, phone(s), email, home value, hail size, tipo, source, created date
Pipeline	Current status, full status change history with user and timestamp
Assignment	Assigned Rufero, assigned by, date — reassignment button for Admin
Calls	All call records: date, time, agent, duration, disposition, recording playback button
SMS	Full threaded SMS conversation
Email	All emails sent and received with subject, body, timestamps
Appointments	All appointments: date, time, Rufero, status, notes
Documents	All documents: type, created, status (unsigned/signed), download link
Inspection	All field inspection records with photos, damage form data, AI analysis (if available)
Activity	Complete chronological audit log of every action on this record
Notes	Team notes with author and timestamp — add new note button
Map	Mini embedded map showing property location with Street View link

7.4 Prospect Data Model

Field	Type	Notes
id	String	Firestore auto-ID
tenantId	String	REQUIRED — partition key for all queries
name	String	Homeowner full name
address	String	Street address
city	String	City name
state	String	State code (TX, AR, etc.)
zip	String	ZIP code
coordinates	GeoPoint	Lat/lng for map
geohash	String	For proximity search queries
phone	Array	Multiple phone numbers supported
email	String	Homeowner email
homeValue	Number	USD estimated value
hailSize	Number	Hail size in inches
status	String	Current pipeline status
tipo	String	Record type label
source	String	manual / storm-import / ai-agent / csv-import / api

assignedTo	String	Rufero user ID
assignedBy	String	Admin user ID
assignedAt	Timestamp	When assigned
doNotCall	Boolean	DNC flag — disables Call and SMS
doNotCallReason	String	Reason for DNC
doNotCallAt	Timestamp	When DNC was set
tags	Array	Custom tags
notes	Array	{text, authorId, timestamp}
createdAt	Timestamp	Creation timestamp
updatedAt	Timestamp	Last update timestamp
createdBy	String	Creating user ID

7.5 DNC Compliance — CRITICAL

TCPA compliance is a legal requirement. Violating DNC regulations can result in fines of \$500-\$1,500 per call. The system must enforce this.

- DNC flag disables Call and SMS buttons — cannot be bypassed without explicit override + reason
- Auto-DNC: SMS STOP reply automatically flags the record
- Auto-DNC: Homeowner says ‘remove me’ during AI call — AI flags the record
- DNC records never deleted — permanent flag with full audit trail
- CSV import must include DNC check against existing records
- National DNC Registry check: integration stub required — checkDNCRegistry(phone) Cloud Function
- Calling hours enforcement: system prevents calls outside configured hours (e.g. 8am–8pm local time)

8. Communication Modules (Phone / SMS / Email)

8.1 Phone Module — Human Softphone

WebRTC Softphone UI:

- Microphone device selector (auto-detects all connected audio inputs)
- Voice level meter — real-time visual indicator
- Number input field — manual dial or click-to-call from prospect card
- CALL button, MUTE, HOLD, TRANSFER, HANG UP in-call controls
- Incoming call banner: Caller ID, Accept and Reject buttons

- Connection status: green Ready / red Error in top navigation bar with reconnect button
- Debug log panel (collapsible) for troubleshooting

Call Features:

- Click-to-call from any prospect card or profile — number pre-populated
- Outbound Caller ID = agent's personal Telnyx extension number
- Inbound routing: Telnyx webhook → identify tenant → match extension → route to agent WebRTC session
- Call recording: all calls recorded to Firebase Storage at {tenantId}/call-recordings/{callId}.mp3
- Recording disclosure: configurable per-tenant message played at call start (required in many US states)
- Voicemail detection: option to drop a pre-recorded message if voicemail detected
- Call disposition prompt: after hangup, agent selects outcome: Answered / No Answer / Voicemail / Wrong Number / DNC Request / Callback Requested
- All calls logged in call_logs collection: prospectId, agentId, tenantId, direction, duration, disposition, recordingUrl, timestamp

Telnyx Architecture:

- One main number provisioned per tenant
- Personal extension per Telefonista (configurable count per plan)
- Call Control Application configured per tenant with tenant-specific webhook URL
- WebRTC session credentials generated per-user per-session via Cloud Function

8.2 SMS Module

- Personal extension number used as sender per agent
- Two-way threaded conversation view per prospect
- SMS templates: pre-written messages selectable and personalizable
- Character count and segment counter (1 segment = 160 chars)
- Delivery status: Sent / Delivered / Failed per message
- Auto opt-out: STOP keyword → prospect DNC flagged + confirmation message sent
- MMS support: agents can attach images (e.g. PDF link, property photo)

- Inbound routing: Telnyx webhook → match number → route to correct agent inbox
- All messages saved to sms_logs: prospectId, agentId, tenantId, direction, body, status, timestamp

8.3 Email Module

- Personal sender per agent: firstname.lastname@{company}.roofaid.app via SendGrid Subuser
- Reply-to: agent's personal address — replies go directly to the agent
- Compose: To (pre-filled), Subject, Body (rich text with formatting)
- Email templates per document type and use case — Admin configures templates
- Inbound sync: SendGrid inbound parse webhook OR IMAP polling every 5 minutes
- Bounce handling: SendGrid bounce webhook marks prospect email as invalid
- Spam complaint: prospect flagged in Firestore — no further marketing emails sent
- CAN-SPAM compliance: unsubscribe link in all bulk emails; physical address in footer
- All emails logged: prospectId, agentId, tenantId, direction, subject, body, status, timestamp

8.4 Internal Notification System

- Bell icon in navigation shows unread notification count — real-time via Firestore listener
- Notification types: new appointment assigned, document signed, inbound call/SMS/email, lead assigned, AI call completed, system alert
- Web push notifications via Service Worker (browser must grant permission)
- Mobile push via FCM — see Section 12
- Notification preferences per user: toggle each type on/off
- Notifications stored in notifications Firestore collection: userId, tenantId, type, message, relatedId, read, createdAt

9. Documents & Contracts Module

9.1 Document Types

Type	Description	Timing
3rd Party Authorization	Authorizes roofer to deal with homeowner's insurance on their behalf	Before inspection — required to open a claim

ACV Contract	Actual Cash Value — homeowner accepts depreciated insurance payout	When homeowner has ACV-only policy
RCV Contract	Replacement Cost Value — full replacement cost covered	When homeowner has full RCV coverage
Supplement	Additional insurance claim items not in original estimate (Tier 4+)	After initial claim, to recover full costs

9.2 Document Generation Workflow

1. Step 1: User navigates to Generate Contract or opens from prospect profile
2. Step 2: Selects prospect from assigned list (web) or views current prospect (mobile)
3. Step 3: Clicks New Document button
4. Step 4: Selects document type from dropdown
5. Step 5: Clicks Continue
6. Step 6: Cloud Function generates PDF from template, pre-populated with: homeowner name, address, company name, date, agent name, company contact info, property details
7. Step 7: PDF stored at {tenantId}/documents/{prospectId}/{docId}.pdf in Cloud Storage
8. Step 8: Document record created in documents collection: prospectId, tenantId, type, status=generated, storagePath, createdBy, createdAt
9. Step 9: Document available for: on-screen signing, email delivery to homeowner, or download/print

9.3 E-Signature Flow

10. 1: Agent opens document on device (mobile app or web)
11. 2: Document displayed as scrollable PDF preview
12. 3: Signature pad shown at bottom — homeowner signs with finger or stylus
13. 4: Clear and Confirm buttons shown
14. 5: On Confirm: signature image (SVG or PNG) sent to Cloud Function
15. 6: Cloud Function embeds signature into PDF at designated position
16. 7: Signed PDF saved to Storage, document record updated: status=signed, signedAt, signedBy, signatureUrl
17. 8: Admin and Telefonista notified in real time

18. 9: Signed PDF automatically emailed to homeowner

9.4 Document Security

- All documents only accessible by authenticated users of the same tenant
- Download URLs are signed (time-limited: 1 hour expiry) — never publicly guessable
- Original unsigned version preserved — signed version stored separately (version history)
- Certified flag on signed documents for easy filtering
- Admins can delete documents — action logged and requires confirmation

10. Appointments & Scheduling Module

10.1 Appointment Statuses

Status	Description
Pending	Created, homeowner not yet confirmed
Confirmed	Homeowner confirmed date and time
Completed	Inspection conducted
Cancelled	Cancelled by either party — reason logged
No-show	Rufero arrived, homeowner absent — triggers auto follow-up
Rescheduled	Moved — linked to original appointment

10.2 Scheduling Workflow

19. 1: Telefonista confirms interest during call
20. 2: Opens appointment scheduler from prospect card
21. 3: Calendar shows Rufero availability (no overlapping appointments)
22. 4: Selects date/time, assigns Rufero (or auto-assign by proximity)
23. 5: Appointment created, prospect status — Scheduled (Appointment)
24. 6: Rufero gets push notification on mobile
25. 7: Homeowner receives SMS confirmation
26. 8: Automated SMS reminders: 24h before and 2h before (via scheduled Cloud Function)
27. 9: Day of: Rufero uses mobile app to navigate, conduct inspection, upload photos, get signature
28. 10: Status updated to Completed — Admin notified

10.3 Auto-Assignment Logic

- When creating appointment, system calculates distance from each active Rufero to the prospect address
- Filters for Ruferos with no conflicting appointment within 2 hours of the requested time
- Suggests closest available Rufero — Admin can override
- Rufero home base address stored in user profile for distance calculations

10.4 Calendar Views

- Month view, week view (hour-by-hour grid), day view
- Filter by Rufero — show individual or all Ruferos
- Color coding by appointment status
- Google Calendar sync — optional per user — appointments pushed to personal Google Calendar
- Rufero can mark unavailable dates/time blocks in their profile

11. Admin, Analytics & Reporting Module

11.1 Admin Panel Sections

Section	Description
User Management	Create, edit, deactivate users. Assign roles, Telnyx extensions, personal email. Force password reset.
Prospect Assignment	Bulk-assign by city, status, or zone. Manual reassign. View unassigned leads.
Activity Log	Real-time feed of all system events — filterable by user, prospect, action type, date range.
Lead Import	CSV upload with field mapping UI. Duplicate detection. Preview before committing import.
City Management	Add/remove cities in the city dropdown.
Document Templates	Upload base PDF templates for each document type. Version management.
Message Templates	SMS and email templates for agents. Create, edit, activate/deactivate.
Calling Hours	Set allowed outbound calling hours per day of week, per tenant timezone.
Recording Disclosure	Configure per-state call recording disclosure message text.
Tenant Settings	Company name, logo, address, timezone, service area.
Feature Flags (Read-Only)	View active features based on current subscription tier.
Billing	View plan, payment history, upgrade/downgrade, invoice

	download. Owner only.
API Keys	Generate API access tokens for external integrations. Show/hide key. Revoke.

11.2 Analytics Dashboard

Metric	Description
Pipeline Funnel	Visual funnel: leads → prospects → contacted → scheduled → closed. Count and % at each stage.
Conversion Rate	% of New Leads that become Closed Customers — by week/month/quarter.
Calls Made	Total outbound calls — by agent, by day, trend chart.
Call Answer Rate	% of calls answered vs voicemail vs no answer.
Average Call Duration	By agent and overall.
Appointments Set Rate	Calls per appointment set — efficiency metric.
Appointments Completed	Completion vs cancelled vs no-show breakdown.
No-Show Rate	% homeowners absent — flags scheduling quality issues.
Close Rate	Inspections-to-contracts ratio — by Rufero.
Documents Signed	Count by type and period.
Supplement Value	Total claim value processed, commissions owed (Tier 4).
Agent Leaderboard	Telefonistas ranked by calls, appts set, and closes.
Revenue Pipeline	Estimated value of active pipeline based on home values and stage.
Lead Source Breakdown	Manual vs storm-import vs AI agent vs API.

11.3 Activity Logging — Required Events

- Prospect: created, status changed (old → new, user, timestamp), deleted, merged
- Assignment: prospect assigned / reassigned to Rufero
- Call: started, answered, ended (duration, recording URL, disposition)
- SMS: sent, received, delivery status update
- Email: sent, received, bounced
- Appointment: created, confirmed, completed, cancelled, no-show
- Document: created, signed, downloaded, deleted
- Inspection: created, photos uploaded, signature captured
- Supplement: created, submitted, commission calculated, paid, disputed

- Import: completed (record count, duplicates skipped)
- User: login, logout, created, role changed, deactivated, password reset
- DNC: flag set, override attempted (approved/rejected), auto-set by STOP reply
- Billing: subscription changed, payment succeeded, payment failed

12. Mobile Application (iOS & Android)

The mobile app is for Rufero (field inspector) users. Built in React Native — one codebase for iOS and Android. Must be production-quality and pass App Store and Play Store review. Same Firebase backend as web CRM.

12.1 App Screens

Screen	Description
Splash / Login	App icon splash, then Firebase Auth login. Biometric (Face ID / fingerprint) on subsequent sessions.
My Schedule	Today's and upcoming appointments chronologically. Date navigation. Tap to open detail.
Appointment Detail	Prospect info, date/time, address, notes. Navigation button. Status update (Confirm / Complete / No-show / Cancel).
My Prospects	All assigned prospects. Search by name or address. Filter by status.
Prospect Detail	Full profile tabs: Overview, Calls, SMS, Appointments, Documents, Inspection, Notes.
Inspection (Core Screen)	Take photos, fill damage form, view previous photos, capture signature, update status.
Photo Capture	Full-screen camera. Select photo type from tag list. GPS and timestamp auto-captured.
Damage Form	Roof age, material type, storm date, affected areas checklist, severity scale, scope notes.
Signature Capture	Full-screen signature pad. Homeowner name and date shown. Clear / Confirm buttons.
Document Viewer	View signed PDFs in-app. Share button (email or AirDrop/Android share sheet).
SMS Inbox	Threaded conversations. Reply from app. Limited to assigned prospects.
Notifications	Push notification inbox. Tap navigates to relevant record.
Settings	Profile, photo, notification preferences, biometric toggle, app version, logout.
SMS Inbox	Threaded two-way SMS for assigned prospects only. Unread count badge. Reply from app. Phase 2 required.
Document Viewer	PDF viewer with pinch-zoom. Share sheet (email/AirDrop).

	Download to device. Shows type, signed date, homeowner.
Appointment Detail	Full detail card: homeowner, address, time, Telefonista notes. Status buttons: Confirm/Complete/No-show/Cancel. Navigate, Call, SMS homeowner. Notes field.

12.2 Offline Mode

Ruferos work in areas with poor or no signal. Offline capability is mandatory, not optional.

- Assigned prospects cached locally via Firestore offline persistence
- Appointment details cached for offline viewing
- Photos taken offline stored in device queue — uploaded when connection restored
- Status updates and notes queued offline — synced on reconnect with conflict resolution
- Signature captures stored locally — uploaded on reconnect
- Offline indicator in app header when no connection detected
- Sync status shown: 'Syncing 3 items...' → 'All synced'

12.3 Photo Management Requirements

Requirement	Detail
Camera permission	Requested on first use following iOS/Android OS guidelines
Photo types	Tags: Overview, Front, Back, Left Side, Right Side, Close-up Damage, Gutters, Chimney, Skylights, HVAC, Siding, Evidence, Other
Auto-metadata	Each photo tagged with: prospectId, inspectionId, agentId, timestamp, GPS coordinates, photoType
Compression	Max 2MB per photo before upload — quality preserved, storage optimized
No limit	Inspector can take unlimited photos per inspection
Storage path	{tenantId}/inspections/{inspectionId}/{photoId}.jpg
Viewer	In-app viewer with pinch-zoom, swipe navigation, type label display
CV ready	Each photo Firestore doc has cvAnalysis: null — ready for computer vision results

12.4 Technical Stack (Mobile)

- React Native — single codebase iOS and Android
- Firebase SDK for React Native — Auth, Firestore, Storage, FCM
- React Navigation v6+ for screen navigation

- Redux Toolkit or Zustand for state management
- react-native-camera or expo-camera for photo capture
- react-native-signature-canvas for signature capture
- React Native Maps with Google Maps provider
- react-native-fs or expo-file-system for offline file queuing
- react-native-biometrics for Face ID / fingerprint login

13. Storm Leads Module (Tier 2)

After a hail storm, homeowners in the impact zone become high-value prospects. This module automatically identifies those properties and imports them as leads into the tenant's CRM.

13.1 Lead Provider Integration

- Data source: TBD — CoreLogic, Tomorrow.io, Verisk, or equivalent
- Provider abstracted behind a LeadProvider interface: `getStormEvents(area, dateRange)` and `getAffectedProperties(stormId)`
- Swapping providers requires only a new adapter, not changes to core system
- Storm event data: storm ID, date, affected area (polygon or radius), max hail size
- Property data per storm: name, address, coordinates, home value, hail size at that location

13.2 Lead Import Flow

29. 1: Storm event detected in tenant's service area by scheduled Cloud Function (runs every 6 hours)
30. 2: If new storm in service area: fetch affected properties from lead provider
31. 3: Deduplicate against existing prospects (match by address)
32. 4: Create new prospect records with source=storm-import, status=New Leads, hailSize from storm data
33. 5: Admin receives notification: 'New storm — 47 leads imported for Springdale area'
34. 6: Leads appear on dashboard map immediately

13.3 Map Radar Overlay

- Radar On/Off toggle in dashboard header
- When Radar On: storm impact zone polygon shown as translucent overlay on map

- Storm date and hail size shown in overlay tooltip
- Multiple storms shown with different colors if overlapping dates
- Tenants can see which of their prospects fall within a storm zone

13.4 Service Area Configuration

- Each tenant configures their service area in Admin Settings: list of cities and/or a geographic radius from a center point
- Storm import only runs for properties within the configured service area
- Tenant can manually trigger a storm search for a specific date and area

14. AI Calling Agent (Tier 3)

The AI Calling Agent automatically dials leads, conducts a qualifying conversation, and schedules an inspection appointment — with no human involved. Built on Telnyx Call Control + LLM (no third-party AI calling platform).

14.1 Architecture

- Telnyx Call Control API handles call setup, media streams, and webhooks
- Real-time audio piped to LLM (GPT-4 or Claude) via media streaming — speech-to-text → LLM → text-to-speech
- AI uses the tenant's personal Telnyx main number — homeowner sees a local caller ID
- AI introduces itself as calling on behalf of [Company Name] — transparent, not deceptive
- Conversation flow: Introduction → Qualify storm damage interest → If interested: offer inspection → Schedule date/time → Confirm → End call
- Human handoff: if homeowner requests a human, AI places them on hold and transfers to available Telefonista

14.2 AI Call Triggers

- Manual: Admin or Telefonista clicks Activate AI Caller on a specific prospect
- Bulk manual: Admin selects multiple New Leads and triggers AI calls in batch
- Automatic (configurable): when a storm lead is imported, AI caller starts automatically if `auto_call=true` in tenant settings
- Calling hours enforced: AI only calls within tenant-configured hours — never outside
- Respects DNC flags: AI cannot call a DNC-flagged record under any circumstances

14.3 Voicemail & Retry Logic

- Voicemail detection: if voicemail detected, AI drops a pre-recorded professional message
- Voicemail message template configurable per tenant
- Retry schedule: configurable — e.g. 3 attempts: day 1 at 10am, day 2 at 2pm, day 3 at 6pm
- Maximum attempts configurable per tenant (default: 3)
- After max attempts with no contact: prospect status stays Prospects with note 'AI: Max attempts reached'

14.4 Post-Call Actions

- Full transcript stored in call record
- Call outcome logged: Answered-Interested / Answered-NotInterested / Answered-DNR / Voicemail / NoAnswer
- If interested: appointment automatically created with preferred date/time extracted from conversation
- If not interested: prospect status updated to Not Viable with AI note
- If DNC requested: immediate DNC flag set
- Agent notification: 'AI call completed — Gary Ahrens: Appointment scheduled for Tuesday 2pm'

14.5 Integration Stubs (Required Even Without AI Model)

- `initiateAICall(prospectId, tenantId)` — Cloud Function — returns `callId`
- `processAICallWebhook(event)` — HTTP endpoint for Telnyx webhooks
- `transferToHuman(callId, tenantId)` — transfers live AI call to Telefonista
- `scheduleFromAICall(prospectId, dateTime, tenantId)` — creates appointment from AI conversation
- `getAICallTranscript(callId)` — returns full transcript

15. Insurance Supplements & Commission Billing (Tier 4)

Insurance supplements are additional claims submitted after the initial insurance estimate, requesting payment for items the adjuster missed or undervalued. Writing them accurately is specialized work. This module uses ML to assist — and Roof-Aid earns 10% of each supplement's claim value.

15.1 Supplement Workflow

35. 1: Rufero completes inspection and uploads photos

36. 2: Telefonista or Admin opens Supplement tool from prospect profile
37. 3: System pre-fills supplement with: property info, inspection photos, damage form data
38. 4: ML model (stub initially) analyzes inspection data and suggests line items: materials, labor, code upgrades, permit fees, depreciation items
39. 5: Agent reviews, edits, and approves the AI-generated line items
40. 6: Agent enters total claim value
41. 7: System calculates commission: $\text{claimValue} \times 10\%$ — shown to agent for transparency
42. 8: Agent clicks Submit Supplement
43. 9: Supplement PDF generated and delivered to agent for filing with insurance company
44. 10: Commission transaction record created in Firestore: `tenantId`, `supplementId`, `claimValue`, `commissionAmount`, `status=pending`
45. 11: Commission billed to tenant via Stripe at month-end (or immediately — configurable)

15.2 Commission Billing Flow

Automatic Monthly Billing:

- On the 1st of each month, a scheduled Cloud Function runs
- Queries all `supplement_commissions` where `status=pending` and `createdAt` in previous month
- Creates a Stripe invoice per tenant with all pending commissions as line items
- Sends invoice to tenant's billing email
- On Stripe payment confirmation: commissions updated to `status=paid`
- On Stripe payment failure: commissions stay pending; tenant flagged for follow-up

Commission Statement:

- Monthly PDF statement auto-generated: lists each supplement, claim value, 10% commission, total due
- Available in tenant's Billing section and emailed automatically
- Super Admin sees all commissions across all tenants: total processed, collected, outstanding

Dispute Process:

- Tenant can flag any commission as disputed with a reason

- Status changes to disputed — billing paused on disputed items
- Super Admin reviews and either approves credit or rejects dispute
- Resolution logged with notes

15.3 Supplement Data Model

Field	Type	Description
id	String	Firestore auto-ID
tenantId	String	Tenant isolation key
prospectId	String	Related prospect
inspectionId	String	Related inspection record
lineItems	Array	Each: {description, quantity, unit, unitPrice, total}
claimValue	Number	Total supplement claim amount in USD
commissionAmount	Number	claimValue × 0.10
status	String	draft / submitted / accepted / denied / revised
commissionStatus	String	pending / billed / paid / disputed / credited
mlGenerated	Boolean	True if AI-suggested line items were used
mlConfidence	Number	0-1 confidence score from ML model
supplementPdfUrl	String	Firebase Storage URL
submittedAt	Timestamp	When submitted to insurance
createdBy	String	User ID
createdAt	Timestamp	

15.4 ML Supplement Stub

- generateSupplement(inspectionId, tenantId) — Cloud Function stub — returns { lineItems[], estimatedClaimValue, confidence }
- When ML model is ready, stub implementation replaced — no other code changes needed
- Stub returns: empty lineItems with placeholder message ‘AI analysis not yet available’

16. AI Roof Inspection — Computer Vision (Tier 5)

Computer vision analyzes roof photos uploaded during inspections to automatically assess damage severity, identify affected areas, and generate a damage report. The infrastructure must be ready at launch even though the AI model is built separately.

16.1 Architecture Ready Now

- All inspection photos stored with consistent metadata: prospectId, inspectionId, photoType, GPS, timestamp
- Each photo document in Firestore has field cvAnalysis: null — populated when AI runs
- Cloud Function stub: analyzeRoofPhotos(inspectionId, tenantId) — accepts inspection, returns damage assessment
- Damage assessment schema defined now: { overallCondition: String, damageScore: Number (0-100), affectedAreas: Array, estimatedAge: Number, estimatedDamage: String, recommendedAction: String, confidence: Number, analyzedPhotos: Array, analyzedAt: Timestamp }
- Mobile app has placeholder AI Analysis card in Inspection screen — shows 'Analysis pending' if cvAnalysis is null
- When CV is activated: stub function gets real implementation — zero other code changes

16.2 Future CV Capabilities (Design for These)

- Damage detection: identify hail damage, wind damage, missing shingles, granule loss, flashing issues
- Severity scoring: 0-100 damage score based on photo analysis
- Area mapping: which sections of the roof are affected (front, rear, left, right, ridge, valleys)
- Age estimation: estimated roof age from photo characteristics
- Repair recommendation: repair / partial replacement / full replacement
- Auto-populate supplement line items based on CV findings
- Insurance-grade damage report PDF generated automatically

17. Multi-Tenant Architecture

This is the most critical architectural requirement. Every database record, every query, every Cloud Function, and every security rule must be tenant-aware. Getting this wrong breaks security for all customers.

17.1 Tenant Isolation Rules — Non-Negotiable

These rules must be enforced at the DATABASE LEVEL via Firestore Security Rules — frontend checks alone are insufficient.

- tenantId is the first field in EVERY composite Firestore index
- Every Firestore Security Rule verifies tenantId matches the authenticated user's tenantId JWT claim

- No query ever returns data without a tenantId filter — no exceptions
- Cloud Functions validate tenantId from JWT token before processing any request
- Storage paths prefixed with tenantId: {tenantId}/documents/, {tenantId}/photos/, etc.
- No cross-tenant data access is possible from any code path

17.2 Tenant Data Model

Field	Type	Description
id	String	Unique tenant ID — used as partition key everywhere
companyName	String	Roofing company name
domain	String	Custom subdomain: roofpro-austin.roofaid.app
subscriptionTier	Number	1/2/3/4/5
features	Object	Feature flags object (see Section 3.2)
telnyx.mainNumber	String	Tenant's main phone number
telnyx.extensions	Array	User extension assignments
telnyx.callControlAppId	String	Telnyx Call Control Application ID
sendgrid.subuser	String	SendGrid subuser for email
sendgrid.sendingDomain	String	Email sending domain
stripe.customerId	String	Stripe customer ID for billing
stripe.subscriptionId	String	Active Stripe subscription ID
serviceArea.cities	Array	Cities in service area
serviceArea.center	GeoPoint	Center of service area for geo search
serviceArea.radiusMiles	Number	Service area radius
settings.timezone	String	Tenant timezone (e.g. America/Chicago)
settings.callingHours	Object	Per-day allowed calling windows
settings.recordingDisclosure	String	State-specific recording disclosure text
status	String	active / trial / suspended / cancelled
trialExpiresAt	Timestamp	End of free trial
createdAt	Timestamp	Onboarding date
billingEmail	String	Where invoices are sent
ownerUserId	String	Owner user ID

17.3 Automated Tenant Provisioning

New tenant onboarding must complete in under 2 minutes via a single Cloud Function call (hard performance target validated against the production v1 system baseline):

46. 1: Create tenant document in Firestore with provided company details
47. 2: Create owner user in Firebase Auth with custom claims: {tenantId, role: 'owner'}
48. 3: Provision Telnyx phone number and Call Control Application for the tenant
49. 4: Create SendGrid Subuser and configure sending domain
50. 5: Create Stripe customer and subscribe to selected plan
51. 6: Configure custom subdomain in Firebase Hosting
52. 7: Seed default data: city list, document templates, message templates
53. 8: Send welcome email with login credentials and getting-started guide

17.4 Super Admin Console

Separate interface (web) accessible only to Roof-Aid platform owner:

- All tenants list: name, plan, status, user count, MRR, created date
- Tenant detail: all config, user list, usage stats, billing history
- Provision new tenant (manual or triggered from website signup)
- Suspend / unsuspend / cancel tenant
- Override feature flags per tenant
- Platform-wide analytics: total tenants, MRR, commission revenue, active users
- System health: Cloud Function errors, latency, cost per tenant
- Commission management: all supplement commissions across all tenants

18. Billing, Subscriptions & Payment Processing

Stripe is the payment processor. The billing system must handle subscription management, supplement commission collection, invoicing, dunning (failed payment recovery), and the full customer billing lifecycle.

18.1 Stripe Integration

- Stripe Customers: one Stripe customer per tenant, created during provisioning
- Stripe Products/Prices: one product per tier — created in Stripe dashboard, referenced by Price ID in platform_config
- Stripe Subscriptions: tenant subscribed to their tier's price — recurring monthly or annual

- Stripe Invoices: supplement commissions added as invoice line items monthly
- Stripe Webhooks: `payment_intent.succeeded`, `customer.subscription.updated`, `invoice.payment_failed`, `invoice.payment_succeeded`, `customer.subscription.deleted`
- PCI compliance: Stripe Elements or Stripe Checkout handles card collection — card data never touches Roof-Aid servers

18.2 Subscription Lifecycle

Event	System Action
New signup	Create Stripe customer, subscribe to plan, set trial period if applicable
Trial ends	Stripe auto-charges first payment — if fails, grace period starts
Payment succeeds	Subscription active, features fully enabled, invoice record created
Payment fails (1st)	Email notification to Owner, grace period starts (7 days)
Payment fails (grace period end)	Account suspended — features disabled, data preserved, login shows payment required screen
Payment updated	Account reactivated immediately
Upgrade	Stripe subscription updated, prorate charged, new features enabled immediately via feature flags
Downgrade	Stripe subscription updated at next billing cycle, features disabled at cycle end
Cancellation	Subscription cancelled at period end, account deactivated, data export offered
Reactivation	New subscription created, data restored, features re-enabled

18.3 Customer Billing Portal

- Self-service billing portal for Owners — powered by Stripe Customer Portal
- Update payment method (credit card, bank account)
- View and download all invoices
- View supplement commission history and monthly statements
- Upgrade/downgrade subscription
- Cancel subscription (with exit survey)
- Update billing email and address

18.4 Dunning & Recovery

- Automated email sequence on payment failure: Day 1 (failure notice), Day 3 (reminder), Day 7 (final warning with suspension notice)
- Email links directly to billing portal for easy payment update
- Stripe Radar for fraud detection
- Smart retry: Stripe automatically retries failed charges on optimal days

19. Website, Marketing & Acquisition Integrations

The marketing website is the front door to the SaaS. It must capture leads, convert them to trials, and feed them automatically into the platform's onboarding flow.

19.1 Marketing Website (roofaid.app)

- Homepage: hero, value proposition, features, tier comparison, testimonials, CTA
- Pricing page: clear tier table, monthly/annual toggle, Start Free Trial buttons
- Features pages: dedicated page per major module
- Blog/Resources: SEO content targeting roofing industry keywords
- Case studies and testimonials from existing customers
- Live chat widget (Intercom, Crisp, or equivalent) — connected to customer service system
- Demo request form — triggers automated email sequence and assigns to sales
- Status page: platform health and uptime transparency (statuspage.io or similar)

19.2 Self-Signup Flow

54. 1: Visitor clicks Start Free Trial on pricing page
55. 2: Signup form: company name, owner name, email, password, phone, service city, select tier
56. 3: Email verification sent — must verify before proceeding
57. 4: On verification: automated tenant provisioning triggered (< 5 minutes)
58. 5: Welcome email sent with login URL, quick-start guide, and video walkthrough link
59. 6: Owner lands on CRM for the first time — onboarding checklist shown (see Section 20)
60. 7: Trial clock starts — 14 days
61. 8: Trial day 7: check-in email with tips and upgrade offer

62. 9: Trial day 12: urgency email with offer to extend trial if needed
63. 10: Trial day 14: trial expired — payment required or data held for 30 days

19.3 CRM Integrations (External)

Integration	Purpose	Priority
Zapier / Make (Integromat)	Connect Roof-Aid to thousands of other tools via webhooks — REST API required	High
Google Analytics 4	Track website conversions and funnel drop-offs	High
Meta Pixel	Retargeting ads for trial signups and upgrades	Medium
Google Ads	Conversion tracking for paid search campaigns	Medium
Intercom / Crisp	Live chat and in-app messaging for customer support	High
Stripe	Payment processing — already in core platform	Critical
Google Calendar	Rufero appointment sync	Medium
Xactimate / Symbility	Insurance estimating platforms — supplement data export (stub)	Medium (Future)
EagleView / GAF	Aerial roof measurement data import (stub)	Low (Future)
Quickbooks / Xero	Accounting integration for invoice sync (stub)	Low (Future)

19.4 REST API (for External Integrations)

- REST API available to Tier 5 customers or as add-on (feature flag: features.apiAccess)
- Authentication: API key per tenant, generated in Admin panel
- Endpoints: GET/POST /prospects, GET/POST /appointments, GET /documents, POST /leads, GET /analytics
- Rate limiting: 100 requests/minute per API key
- API documentation: auto-generated via OpenAPI spec
- Webhook outbound: tenants can configure webhooks to receive events (new lead, appointment created, document signed)

20. Customer Onboarding & Self-Service

20.1 In-App Onboarding Checklist

When a new Owner logs in for the first time, an onboarding checklist is shown as a persistent panel until all steps are complete:

1. 1: Add your company logo and address (Admin Settings)
2. 2: Add your first team member (User Management)
3. 3: Import your first leads (Lead Import or manual add)
4. 4: Make your first call (Phone module)
5. 5: Schedule your first appointment
6. 6: Generate your first contract (Generate Contract)
7. 7: Download the mobile app (Rufero)

Progress shown as: 3 of 7 complete. Each step has a direct action button. Checklist dismissible after 7/7 complete.

Phase 1 required: This checklist is a required Phase 1 UI component shown as a persistent panel on the dashboard for new Owner accounts. It must be built and tested before first customer onboarding.

20.2 Help & Documentation

- In-app help icon (?) on every screen — links to relevant help article
- Help center: searchable knowledge base (Intercom Articles, Notion, or equivalent)
- Video walkthroughs: 2-5 minute videos for each major module
- Getting started guide: PDF and web version sent in welcome email
- Keyboard shortcuts guide for power users
- Changelog: in-app notification when new features are released

20.3 Self-Service Account Management

- Password reset via email — standard Firebase Auth flow
- Update profile: name, photo, email, phone
- Update company settings: logo, address, timezone, calling hours
- Add/remove users without contacting support
- Upgrade/downgrade plan without contacting support
- Export all data: prospects, documents, call logs (GDPR/data portability)
- Delete account: data export offered, then deletion after 30-day window

21. Customer Service & AI Support Agent

Customer support must be excellent — this is a key differentiator for a vertical SaaS. The support system combines self-service resources, an AI support agent, and escalation to human support.

21.1 Support Channels

Channel	Description	Response SLA
In-app AI Chat	AI agent (powered by LLM) answers common questions, troubleshoots, and can perform actions in the account	Instant
Help Center	Searchable knowledge base with articles and videos	Self-serve
Email Support	Human support via support@roofaid.app — tickets tracked in helpdesk	< 4 hours business days
Live Chat	Human agent chat during business hours (Intercom or Crisp)	< 5 minutes during hours
Phone Support	Reserved for Enterprise tier — direct phone line to support team	Immediate
Status Page	Platform health and incident updates at status.roofaid.app	Real-time

21.2 AI Support Agent Requirements

- Integrated as a chat widget in the bottom-right corner of every screen in the CRM web app and every screen of the mobile app. Present in Phase 1 even if the AI model is a stub that escalates to email support
- Powered by LLM (Claude or GPT-4) with a system prompt trained on Roof-Aid documentation, FAQs, and known issues
- Context-aware: AI knows which screen the user is on and can provide relevant help
- Can answer: How do I import leads? Why is my phone not connecting? How do I assign a Rufero? How does supplement billing work?
- Can perform actions with user consent: navigate user to correct screen, pre-fill a form, re-trigger a notification
- Cannot access or display other tenants' data — strictly scoped to current user's tenant
- Escalation: if AI cannot resolve, offers to create a support ticket or connect to human chat
- All AI support conversations logged for quality review and model improvement

21.3 Helpdesk Integration

- Customer support tickets managed in helpdesk tool (Intercom, Freshdesk, or Zendesk)
- Ticket automatically includes: tenant name, plan, user role, browser/device info, and last 5 actions taken in the CRM
- This context means support agents can resolve issues faster without asking basic questions
- Bug reports from support feed into a development backlog

21.4 Error Handling & User Feedback

- All errors shown to users in plain English — never raw error codes
- Error messages include a suggested action: ‘Connection lost. Check your internet and click Reconnect.’
- Error logging: all unhandled errors sent to monitoring (Cloud Logging + Sentry or equivalent)
- User can report a bug from any screen with a single click — includes screenshot and system info

22. Security, Compliance & App Store Requirements

22.1 Authentication Security

- Firebase Auth Email/Password — primary method
- Google OAuth — prepared, enabled in Phase 2
- Multi-factor authentication (MFA) — available for Owner and Admin roles
- Biometric login on mobile (Face ID / fingerprint) via react-native-biometrics
- JWT tokens: 1-hour TTL, 30-day refresh, custom claims: {tenantId, role}
- Force logout: Admins can invalidate all sessions for a user
- Session management: active sessions visible in Admin panel; ability to revoke
- Password requirements: minimum 8 chars, 1 uppercase, 1 number

22.2 Data Security

- Firestore Security Rules enforce tenant isolation at database level — not optional
- Firebase App Check: REQUIRED from Phase 1 — initialize in web app (reCAPTCHA v3 provider) and mobile app (DeviceCheck iOS, Play Integrity Android) before any production deployment
- Rate limiting on all Cloud Functions: prevent abuse and DDoS

- All secrets in Google Secret Manager — never in code, env files, or Firestore
- DOMPurify sanitizes all user-generated content before rendering — prevents XSS
- HTTPS enforced everywhere — Firebase Hosting handles SSL automatically
- CORS configured on all Cloud Functions — only allowed origins can call APIs
- Signed URLs for all file downloads — expire after 1 hour
- Encryption at rest: Firebase/GCP handles AES-256 for Firestore and Storage
- Encryption in transit: TLS 1.2+ on all connections

22.3 Legal & Compliance

Regulation	Requirement
TCPA	Call recording disclosure, DNC compliance, SMS opt-out (STOP keyword), calling hours enforcement
CAN-SPAM	Unsubscribe link in all emails, physical address in email footer, honor opt-outs within 10 days
GDPR	Data export on request, right to deletion, privacy policy, data processing agreement for EU users
CCPA	California users: right to know, right to delete, right to opt-out of sale
HIPAA	Not applicable — no health data collected
SOC 2	Target for Year 2 — Firebase/GCP infrastructure is SOC 2 certified
PCI DSS	Card data handled entirely by Stripe — Roof-Aid is out of scope for PCI

22.4 Apple App Store Requirements

Non-compliance with App Store guidelines results in rejection. These must be implemented before first submission.

- App Store Connect account: developer account registered, certificates and provisioning profiles set up
- Privacy policy URL: required — must cover camera, location, microphone, contacts usage
- App Tracking Transparency (ATT): if any tracking occurs, ATT prompt must be shown before tracking begins
- Privacy manifest: list all SDKs and data types collected — required since iOS 17
- No private APIs: only public Apple APIs used
- Minimum iOS version declared in Info.plist and consistent with App Store listing

- Permission strings in Info.plist: NSCameraUsageDescription, NSLocationWhenInUseUsageDescription, NSMicrophoneUsageDescription — all with clear user-facing explanations
- In-app purchases: if any subscription sold via app, Apple's 30% cut applies — discuss with product owner whether to exclude in-app purchase and direct to website
- Metadata: app name, description, keywords, screenshots for all required device sizes (iPhone and iPad)
- TestFlight beta testing: mandatory before production submission

22.5 Google Play Store Requirements

- Google Play Console account: developer account registered, app signing by Google Play enabled
- Target API level: must target Android 14 (API 34) or latest as required by Play Store policy
- Privacy policy: required URL in Play Console — must cover all sensitive permissions
- Data safety form: complete the Play Console Data safety section — what data is collected, shared, encrypted, deleteable
- Permissions: only declare permissions actually used — no overly broad permission requests
- Permission rationale: if requesting dangerous permissions (camera, location, microphone), show rationale dialog explaining why before requesting
- 64-bit compliance: all native code must be 64-bit compatible
- App Bundle: submit as .aab (Android App Bundle), not .apk
- Content rating: complete the content rating questionnaire in Play Console
- Store listing: title, short description (80 chars), full description, screenshots (phone and tablet), feature graphic
- Internal testing track — Closed testing — Open testing — Production — follow staged rollout

22.6 Penetration Testing & Security Audits

- Basic security audit before first production deployment
- Firestore Security Rules reviewed by a second developer before production
- OWASP Mobile Security Testing Guide checklist completed for the mobile app
- Annual security review recommended post-launch

23. Notifications, Webhooks & Integrations

23.1 Notification Types & Channels

Event	In-App	Email	SMS	Push (Mobile)	Push (Web)
New lead assigned	YES	YES	Optional	YES	YES
Appointment created	YES	YES	YES (to homeowner)	YES	YES
Appointment reminder (24h)	YES	NO	YES (to homeowner)	YES	NO
Appointment reminder (2h)	NO	NO	YES (to homeowner)	YES	NO
Document signed	YES	YES	NO	YES	YES
Inbound call	YES	NO	NO	YES	YES
Inbound SMS	YES	NO	NO	YES	YES
Inbound email	YES	YES	NO	YES	YES
AI call completed	YES	YES	NO	YES	YES
Payment failed	YES	YES	NO	NO	NO
Trial expiring (3 days)	YES	YES	NO	NO	NO
Storm leads imported	YES	YES	NO	YES	YES

23.2 Outbound Webhooks (for Tenant Integrations)

- Tenants can configure webhook URLs in Admin Settings (Tier with API access)
- Events delivered: prospect.created, prospect.statusChanged, appointment.created, document.signed, call.completed, lead.imported
- Payload: JSON with event type, timestamp, tenantId, and full relevant object
- Retry logic: 3 retries with exponential backoff on failure
- Delivery log: each webhook delivery logged with status code and response

23.3 Required Cloud Function Stubs

Function Name	Trigger	Description
initiateAICall	Callable	Start AI outbound call to prospect
processAICallWebhook	HTTP	Handle Telnyx AI call events
transferToHuman	Callable	Transfer live AI call to human agent
generateSupplement	Callable	ML supplement generation
analyzeRoofPhotos	Callable	CV roof damage analysis

importStormLeads	Scheduled / Callable	Import leads from storm data provider
checkDNCRetry	Callable	Check phone against National DNC Registry
sendAppointmentReminder	Scheduled	24h and 2h appointment SMS reminders
processMonthlyCommissions	Scheduled (1st of month)	Create Stripe invoices for supplement commissions
generateMonthlyStatement	Scheduled	Create PDF commission statements
exportTenantData	Callable	GDPR/CCPA data export
deleteTenantData	Callable	GDPR/CCPA account deletion
outboundWebhookDelivery	Firestore trigger	Deliver webhook to tenant endpoint on events
generateMonthlyStatement	Scheduled (1st)	Generate PDF commission statement per tenant
exportTenantData	Callable	GDPR/CCPA: export all tenant data as ZIP archive
deleteTenantData	Callable	GDPR/CCPA: permanently delete tenant data after 30-day hold
sendTrialEmailSequence	Scheduled (daily)	Day 7 tips email, Day 12 urgency email, Day 14 expired email with Stripe Checkout link embedded

24. Business Processes & Operational Workflows

This section documents end-to-end business processes as they will be experienced by users and customers. These are the workflows a developer must understand to build a system that actually works in the real world.

24.1 New Roofing Company Signs Up

- 1: Owner visits [roofaid.app](#) and clicks Start Free Trial
- 2: Fills signup form: company name, name, email, password, phone, primary city, selects tier
- 3: Email verification sent — clicks link in email
- 4: Automated provisioning: account created, phone number provisioned, email configured (< 5 min)
- 5: Owner receives welcome email with login URL, getting-started guide, and quick-start video
- 6: Owner logs in — sees onboarding checklist, imports first leads, adds team members
- 7: 14-day trial — usage monitored; auto-email check-ins at day 7 and day 12

8. 8: Day 14: prompted to add payment method — Stripe Checkout embedded in app
9. 9: Subscription active — recurring charges begin

24.2 Daily Telefonista Workflow

8. 1: Log in — dashboard shows pipeline status overview
9. 2: Select a city and status (New Leads or Prospects) and click Query Database
10. 3: Review prospect cards on map and in list — prioritize by home value and hail size
11. 4: Click Call on a prospect card — softphone dials automatically
12. 5: Call answered: qualify damage interest, check if homeowner is interested in free inspection
13. 6: If interested: click Appt, select date/time, assign Rufero — appointment created
14. 7: Update prospect status to Contacted or Scheduled as appropriate
15. 8: If voicemail: leave message, add note, status stays Prospects for retry
16. 9: If not interested or DNC request: update status to Not Viable or flag DNC
17. 10: End of day: manager sees activity log and leaderboard in analytics

24.3 Rufero Inspection Day Workflow

64. 1: Open mobile app — see today's appointments
65. 2: Tap appointment — see homeowner name, address, notes from Telefonista
66. 3: Tap Navigate — Google Maps opens with turn-by-turn directions
67. 4: Arrive at property — open Inspection in the app
68. 5: Take photos: overview shots, all sides of roof, close-ups of damage areas
69. 6: Fill damage form: roof age, material, affected areas, severity, scope notes
70. 7: If homeowner present: pull up contract (3rd Party Auth or Contract)
71. 8: Homeowner reviews and signs on the phone/tablet screen
72. 9: App uploads signed PDF to Firebase, Admin and Telefonista notified
73. 10: Update appointment status to Completed
74. 11: All data synced — prospect profile updated with inspection, photos, and signed document

24.4 Supplement Filing Process (Tier 4)

75. 1: Rufero completes inspection with full photo documentation
76. 2: Admin or Telefonista opens prospect profile — sees Inspection tab with all photos
77. 3: Clicks Create Supplement
78. 4: ML assistant analyzes inspection data and suggests line items (when model is active)
79. 5: Agent reviews, adjusts quantities and prices, adds any missed items
80. 6: Enters total claim value — system shows: Commission due: \$X (10%)
81. 7: Clicks Submit — supplement PDF generated and downloaded by agent
82. 8: Agent files the supplement with the insurance company (external process)
83. 9: Commission transaction created: pending
84. 10: At month end: Stripe invoice sent to tenant for total commissions
85. 11: Agent marks supplement as accepted/denied based on insurance response

24.5 Charge & Billing Processes

Monthly Subscription Charge:

- Stripe automatically charges on the same day each month
- Invoice emailed to billing email on successful charge
- Failed charge: automated dunning sequence starts (see Section 18.4)

Supplement Commission Charge:

- Monthly scheduled Cloud Function runs on the 1st
- Queries all pending supplement commissions from previous month
- Creates Stripe invoice with line items: 'Supplement Commission: [Property Address] — \$XXX claim — \$XX commission'
- Invoice emailed to tenant billing email with full statement PDF attached
- Stripe charges the payment method on file
- On success: all commissions updated to paid; failure triggers dunning

Trial to Paid Conversion:

- Day 14: in-app modal and email: 'Your trial has ended. Add payment to continue.'
- Stripe Checkout embedded directly in the app — no redirect to external page
- On payment: subscription starts, trial flag removed, full access continues uninterrupted

25. Edge Cases & Scenarios Not to Miss

25.1 Data & Sync Edge Cases

- Duplicate prospects: same address imported twice — deduplicate on import, warn on manual create
- Prospect with multiple phone numbers: all numbers stored; any can be called; DNC applies to all numbers
- Rufero goes offline mid-inspection: all data queued; on reconnect synced in correct order
- Two agents editing same prospect simultaneously: last-write-wins with timestamp; show conflict warning
- Tenant imports 10,000 leads at once: batch write in chunks of 500; progress indicator; do not block UI
- Photo upload fails: retry 3 times automatically; if still failing, show error with manual retry button
- Signed PDF upload fails: signature data preserved locally until upload succeeds; never lose a signature

25.2 Telephony Edge Cases

- Telnyx connection drops mid-call: attempt reconnect, show warning to user, log incident
- Agent closes browser during active call: call continues on Telnyx side; Cloud Function detects orphaned call and ends it after 5 minutes
- Inbound call when agent is already on a call: incoming call banner shown; agent can accept (puts first on hold) or reject
- AI call: homeowner speaks a language other than English: AI detects language and responds in kind (if LLM supports it) or gracefully ends call
- AI call: homeowner becomes abusive or threatening: AI ends call immediately; event logged
- Call recording storage full: monitoring alert fires; admin notified; oldest recordings auto-archived

25.3 Billing Edge Cases

- Tenant upgrades mid-cycle: Stripe prorates the charge for remaining days — display prorate amount before confirming
- Tenant cancels during trial: no charge; data held 30 days; offer to reactivate

- Supplement claim denied: tenant disputes commission; manual review required; credit issued if valid
- Tenant disputes entire month of supplements: commissions set to disputed; billing paused pending review
- Stripe webhook received twice (duplicate): idempotency keys prevent double-processing
- Payment method expires: Stripe sends expiry warning 30 days before; in-app banner and email shown

25.4 Security Edge Cases

- User tries to access another tenant's data via URL manipulation: Firestore rules block at database level; 403 returned
- Deactivated user tries to log in: Firebase Auth disabled; login fails with clear message
- Admin is deleted while logged in: active session continues until token expires (max 1 hour); refresh token revoked
- API key leaked: admin can revoke and regenerate instantly in Admin panel
- Mass data export by disgruntled employee: activity logged; Admin can see large export events in activity log

25.5 App Store Edge Cases

- Apple rejects app: most common reasons pre-handled: privacy policy, permission descriptions, no private APIs. Use Expo EAS build for reproducible builds.
- Google Play policy update: subscribe to Play Console email updates; review policy changes quarterly
- App crashes on specific device: Firebase Crashlytics captures crash reports with device info; fix in next release
- OS update breaks feature: test on beta OS versions before public release using TestFlight and Play internal testing

25.6 Business Continuity

- Lead provider API goes down: cached data still available; new lead import paused; admin notified; retry queue
- Telnyx outage: in-app status banner; calls disabled with clear message; customers notified via email
- SendGrid outage: email queue backed up; retry when service restored; customers notified

- Firebase outage (extremely rare): platform unavailable; status page updated; Firebase SLA compensation applies
- Developer leaves: code must be documented and modular enough that a new developer can onboard in < 1 week

25A. Unresolved Strategic Decisions — Required Before Development Begins

The following decisions directly affect architecture and must be resolved by the product owner before a developer writes a single line of code. Building on the wrong assumption in any of these areas requires expensive rework.

Decision 1: Email Domain Strategy

Two options with different architectural implications:

- Option A — Shared platform domain: All agents send from @company.roofaid.app (e.g., agent@storm-pro.roofaid.app). Lower cost (~\$0/extra), instant configuration, but if any one tenant's user sends spam, the entire platform domain risks blacklisting across all tenants
- Option B — Per-client custom domain: Each roofing company uses their own domain (e.g., ventas@roofpro-austin.com). Full reputation isolation, maximum professionalism. Additional cost to client (~\$12-15/year per domain). Requires DNS setup per tenant during provisioning (adds ~1 minute to onboarding)

Recommended: Option A for launch, Option B as paid upgrade. The SendGrid subuser architecture supports both without code changes.

Decision 2: Telnyx Number Strategy

- Option A — Shared main number + extensions: One main number per tenant + personal extensions per agent. Homeowner sees agent's extension as Caller ID. Cost: ~\$2/month per number + usage. Recommended and already designed into the architecture
- Option B — Dedicated number per tenant user: Each agent gets their own fully independent number. Higher cost but maximum portability. Complicates SMS threading

Recommended: Option A (already built into spec). Confirm this is the final decision.

Decision 3: The 30,000 Existing Prospects

The production v1 system contains approximately 30,000 real homeowner prospect records currently in a single-tenant Firestore database. Before migration, decide:

- Option A — Shared lead pool (Tier 2 feature): The 30k records become platform-owned storm lead data available to all tenants in the relevant geographic areas. This makes them a product differentiator and revenue driver for Tier 2
- Option B — Migrated to founding tenant only: The 30k records belong to the original client (the roofing company that generated them) and migrate into their tenant silo. Other tenants do not see them

- Option C — Split: Records from paid storm lead sources migrate to Tier 2 pool; records entered manually by the original client stay in their silo

This decision cannot be reversed after migration. Choose before development begins.

Decision 4: Telnyx/SMS Billing Model

- Option A — Flat fee included in tier price: Simplest for sales. Risk: high-volume users erode margin. Requires usage caps or fair-use policy
- Option B — Cost-plus markup: The platform operator pays Telnyx at cost; tenants are charged cost + markup per minute/SMS (e.g., cost + 20%). Requires per-tenant usage tracking and a billing line item in Stripe invoices for usage overages

Recommended: Option B with a generous included bundle (e.g., 500 minutes + 1,000 SMS included, overages billed). Requires a usage metering Cloud Function and a usageMetering Firestore collection.

26. Development Build Guide — Step-by-Step

This section is the developer’s primary working document. It defines exactly what to build, in what order, how to use Claude and AI tools throughout, and what “done” looks like before moving to the next step. Nothing is assumed. Nothing should be skipped.

HOW THIS GUIDE WORKS

Each step follows this structure:

WHAT What to build in plain language **WHY FIRST** Why this step must come before the next one **FILES** Exact files to create or modify **AI WORKFLOW** How to use Claude Code and AI agents to build it **DONE WHEN** The specific, testable condition that confirms completion **UNBLOCKS** What becomes possible after this step

Development uses AI-assisted tooling throughout. The primary tools are:

Claude Code For generating, reviewing, and iterating on code. Use it inside the project directory. It reads your files and understands the full context.

Claude claude-sonnet-4-20250514 For architecture decisions, spec interpretation, and reviewing logic before writing code. Ask it questions before you start a step, not after.

AI Agents For repetitive scaffolding: generating boilerplate files, writing test data, running batch operations.

Rule: Before writing any code for a step, paste the relevant spec section into Claude and ask: “What are the top 3 ways I could get this wrong?” Then write the code. Then ask Claude to review it against the spec.

KNOWN BASELINE — WHAT ALREADY EXISTS

A working proof-of-concept (PoC) system has been running in production for 3+ months. It was built as a single-tenant system. Key facts that inform every decision in this build guide:

Stack: Vite + Vanilla JS + Firebase (no React) Bundle size: 580KB gzipped (Firebase 270KB, Telnyx WebRTC 180KB, App code 85KB, DOMPurify 45KB) Performance: FCP 1.2s on 4G, TTI 2.8s on 4G, Lighthouse 92/100 Database: Firestore with 9 collections, 18 composite indexes Functions: 13 Cloud Functions (8 callable, 3 HTTP, 2 scheduled) Cold start: 2.1s (p95), warm execution 180ms (p50), 450ms (p95) Storage: ~6.4GB per client (docs, photos, recordings, templates) Proven cost: ~\$194/month per client at current load Data volume: ~30,000 prospect records proven in production

The new system is a React multi-tenant rebuild. It must match or beat every performance metric above. It does not start from zero — it starts from a proven working system that just needs to be made multi-tenant, modular, and scalable.

PHASE 1 — INFRASTRUCTURE FOUNDATION Goal: A deployed, secure, multi-tenant Firebase project. No features. Just the infrastructure that every subsequent module runs on top of. Target: 2 weeks.

STEP 1 — Firebase Project and Environment Setup

WHAT: Create two Firebase projects (prod and dev), configure all required services, install CLI tooling, and verify the emulator stack runs clean.

WHY FIRST: Every subsequent step deploys to Firebase. If the project setup has mistakes — wrong region, missing APIs, wrong billing plan — they cause cryptic failures 10 steps later. Get this right once.

ACTIONS: 1. Create Firebase project: roof-aid-prod (production) Create Firebase project: roof-aid-dev (development/staging) Both must be in the same Google Cloud organization account.

2. In EACH project, enable these services in order:
 - a. Firebase Authentication — enable Email/Password provider only. Do not enable Google OAuth, phone, or any other provider yet.
 - b. Cloud Firestore — Native Mode, region us-central1. Start in production mode (not test mode).
 - c. Cloud Storage — default bucket, region us-central1.
 - d. Firebase Hosting — single site.
 - e. Cloud Functions — this requires upgrading to Blaze (pay-as-you-go). Do this before writing a single function.

- f. Firebase App Check — in Google Cloud Console.
 - g. reCAPTCHA Enterprise API — in Google Cloud Console. Get the reCAPTCHA v3 site key and store it.
 - h. Cloud Secret Manager API — in Google Cloud Console.
3. Install tooling: `npm install -g firebase-tools` `firebase login` `firebase projects:list` (verify both projects appear)
 4. In the project root, run: `firebase init` Select: Functions, Firestore, Hosting, Storage, Emulators. Functions language: TypeScript. Emulators: Auth, Functions, Firestore, Hosting, Storage.
 5. In Google Cloud Secret Manager, create these secrets in BOTH projects:
`TELNYX_API_KEY` `SENDGRID_API_KEY` `STRIPE_SECRET_KEY`
`STRIPE_WEBHOOK_SECRET` `TELNYX_WEBHOOK_PUBLIC_KEY`
`GOOGLE_MAPS_API_KEY` Add a placeholder value to each. Real values come in Step 8 onwards.
 6. Create `frontend/.env.development` and `frontend/.env.production` with the Firebase config values from each project's console. Never commit these files to version control. Add both to `.gitignore` immediately.

FILES: `firebase.json`, `.firebaserc`, `.gitignore`, `frontend/.env.example`, `frontend/.env.development`, `frontend/.env.production`

AI WORKFLOW: Open Claude Code in the project root after `firebase init` completes. Prompt: "Review my `firebase.json`. I need Functions in `us-central1`, Firestore in `us-central1`, Hosting pointing to `frontend/dist`, and all 6 emulators configured. Show me any mistakes and the corrected file."

DONE WHEN: - `firebase emulators:start` runs with zero errors - All 6 emulators appear in the UI at `localhost:4000` - `firebase use` roof-aid-dev switches the active project cleanly - `.gitignore` contains `.env`, `.local`, `lib/`, `node_modules/`

UNBLOCKS: Step 2 (security rules require a project to deploy to)

—

STEP 2 — Firestore Security Rules and All 23 Indexes

WHAT: Write the complete Firestore security rules enforcing multi-tenant isolation, and all 23 composite indexes. Rules and indexes come before any code that reads or writes data. This is non-negotiable.

WHY FIRST: A developer who writes application code before security rules will inevitably write queries that the rules later block. Write the rules first, then write queries that satisfy them — not the other way around.

KNOWN BASELINE: The PoC uses 18 composite indexes. The multi-tenant system needs all 18 of those plus 5 new ones (the `tenantId` prefix versions). Total: 23 indexes.

ACTIONS: 1. Write firestore/firestore.rules with these helper functions: isAuthenticated() — request.auth != null getUserTenantId() — request.auth.token.tenantId getUserRole() — request.auth.token.role belongsToTenant(tenantId) — authenticated AND token tenantId matches isAdmin() — role in [owner, admin] isTelefonista() — role in [owner, admin, telefonista] Apply tenant isolation to every collection. A user with tenantId=A must never be able to read a document with tenantId=B under any circumstances. This is the fundamental security guarantee.

2. Role permissions by collection: prospects: read (telefonista+, rufero if assignedTo=their uid) write (telefonista+), delete (admin+) appointments: read/write (telefonista+, rufero if their appointment) documents: read (all roles), write (telefonista+, rufero if capturedBy) call_logs: read (all roles), create (authenticated), no delete sms_logs: read (all roles), create (authenticated), no delete supplements: read/write (telefonista+), delete (admin+) commission_transactions: read/write (super_admin only) tenants: read (owner of that tenant, super_admin) write (super_admin only)
3. Write firestore/firestore.indexes.json with all 23 indexes. Every index must have tenantId as the first field. The 23 indexes cover: prospects (7), appointments (3), call_logs (2), sms_logs (2), documents (2), activities (3), notifications (1), supplements (1), commission_transactions (1), users (1).
4. Test rules in the Emulator Rules Playground (localhost:4000/firestore) before deploying. Run these 8 test scenarios:
 - a. User with tenantId=AAA reads prospect with tenantId=BBB → DENY
 - b. Telefonista reads prospect in their tenant → ALLOW
 - c. Rufero reads unassigned prospect → DENY
 - d. Rufero reads prospect assigned to them → ALLOW
 - e. Telefonista deletes a prospect → DENY
 - f. Admin deletes a prospect → ALLOW
 - g. Anyone reads commission_transactions → DENY (unless super_admin)
 - h. Unauthenticated user reads anything → DENY
5. Deploy indexes first (they take 5-10 minutes to build in Firebase): firebase deploy --only firestore:indexes --project roof-aid-dev
6. Deploy rules: firebase deploy --only firestore:rules --project roof-aid-dev
7. In the Firebase console, wait for all 23 indexes to show READY status. Do not proceed to Step 3 until they are all READY.

FILES: firestore/firestore.rules, firestore/firestore.indexes.json

AI WORKFLOW: Prompt Claude Code: "I need Firestore security rules for a multi-tenant CRM. The fundamental rule is: every read and write must verify that the requesting user's tenantId custom claim matches the document's tenantId field. The roles are super_admin, owner, admin, telefonista, rufero. Generate complete rules for collections: prospects, appointments, documents, call_logs, sms_logs, activities, supplements, commission_transactions, notifications,

users, tenants, platform_config. After generating, show me the 8 test scenarios I should run in the emulator playground to verify they are correct.”

DONE WHEN: - All 8 test scenarios pass in the Emulator Rules Playground - firebase deploy – only firestore:rules succeeds with no errors - All 23 indexes show READY in Firebase console (not BUILDING) - No rule allows cross-tenant access under any test scenario

UNBLOCKS: Step 3 (backend code can now write to Firestore safely)

STEP 3 — Backend: TypeScript Scaffold, Config, and Adapters

WHAT: Set up the entire functions/ directory — TypeScript config, dependency installation, secrets layer, Firebase admin singleton, and all 5 service adapters (Telnyx, SendGrid, Stripe, Storage, FCM). No callable functions yet. Just the shared infrastructure layer.

WHY FIRST: Every callable function depends on these adapters. Building adapters in isolation means they can be tested and reviewed before any business logic touches them. It also means the TypeScript compiler catches integration errors at build time, not at runtime.

ACTIONS: 1. Set up functions/package.json with exact dependency versions: firebase-admin ^12.0.0 firebase-functions ^5.0.0 telnyx ^2.0.0 @sendgrid/mail ^8.1.0 stripe ^15.0.0 pdf-lib ^1.17.1 ngeohash ^0.6.3 zod ^3.22.0 Run: npm –prefix functions install

2. Write functions/tsconfig.json: target: es2020, module: commonjs, strict: true, noImplicitReturns: true outDir: lib, sourceMap: true
3. Write functions/src/config/index.ts: Define all secrets using defineSecret() from firebase-functions/params. Define runtime constants: region (us-central1), commissionRate (0.10), trialDays (14), prospectsPerPage (60 — required by Anti-Collision System). Never hardcode API keys. Never hardcode prices. Prices are read from the platform_config Firestore document at runtime.
4. Write functions/src/config/firebase.ts: Single admin.initializeApp() call. Export: db, auth, storage, messaging, FieldValue, Timestamp, GeoPoint, Collections helper object, batchWrite utility function.
5. Write functions/src/adapters/PhoneAdapter.ts (Telnyx): Methods: initiateCall(), hangUp(), transferCall(), getWebRtcCredential(), sendSMS(), purchaseNumber(), assignNumberToApp(), createCallControlApp() All methods must handle Telnyx API errors and throw typed errors.
6. Write functions/src/adapters/EmailAdapter.ts (SendGrid): Methods: sendEmail(), createSubuser(), authenticateDomain(), buildHtmlEmail() (branded template with orange header #E8501F)
7. Write functions/src/adapters/BillingAdapter.ts (Stripe): Methods: createCustomer(), createSubscription(), updateSubscription(), cancelSubscription(), createInvoice(), createBillingPortalSession(), verifyWebhookEvent()

8. Write functions/src/adapters/StorageAdapter.ts: Methods: upload(), getSignedUrl(), delete(), listFiles(), copy() Static paths helper: document(), signedDocument(), photo(), recording(), template()
9. Write functions/src/adapters/NotificationAdapter.ts (FCM): Methods: sendToToken(), sendToUser(), sendToTenant(), saveInAppNotification()
10. Run: npm --prefix functions run build Fix every TypeScript error before proceeding. Zero tolerance.

FILES: functions/package.json, functions/tsconfig.json, functions/src/config/index.ts, functions/src/config/firebase.ts, functions/src/adapters/PhoneAdapter.ts, functions/src/adapters/EmailAdapter.ts, functions/src/adapters/BillingAdapter.ts, functions/src/adapters/StorageAdapter.ts, functions/src/adapters/NotificationAdapter.ts

AI WORKFLOW: Build one adapter at a time. For each adapter prompt Claude Code: “Generate [AdapterName] for [service]. It must: (1) use TypeScript strict mode, (2) handle all API errors with descriptive messages, (3) never use ‘any’ types except where the SDK forces it, (4) export a factory function that creates a new instance using the secret value at call time, not at module load time.” After all adapters are written, prompt: “Review all 5 adapters for missing error handling, incorrect TypeScript types, and any method that a repository will need but I haven’t written yet.”

DONE WHEN: - npm --prefix functions run build completes in under 30 seconds - Zero TypeScript errors or warnings - Zero uses of ‘any’ type except in third-party SDK return types - Each adapter exports a factory function (not a module-level singleton)

UNBLOCKS: Step 4 (repositories depend on adapters and firebase.ts)

STEP 4 — Backend: All Repositories

WHAT: Write all 8 repository classes. Repositories are the only layer that talks directly to Firestore. No other code should call db.collection() directly except repositories.

WHY FIRST: Repositories enforce the tenantId filter on every query. If any callable function queries Firestore directly, it can forget the tenant filter and create a data leak. Centralizing queries in repositories makes that impossible.

ACTIONS: 1. Write TenantRepository.ts: getId(), getByDomain(), getAll(), create(), update(), updateFeatures(), setStatus(), hasFeature(), isActive() The defaultFeaturesByTier constant defines which features each tier unlocks. This is the single source of truth for tier features.

2. Write ProspectRepository.ts: getId(), getPage() (returns 60 records for Anti-Collision), getByProximity() (geohash-based, see Section 29), create(), update(), updateStatus(), setDNC(), addNote(), assign(), bulkCreate() (for CSV/storm import), delete(), isDNC() CRITICAL: getPage() must always include tenantId in the query. CRITICAL: getPage() limit must be 60, not 50.

3. Write AppointmentRepository.ts: getById(), getByTenant(), getByRufero(), getByProspect(), hasConflict() (checks 30-minute buffer around scheduled time), create(), updateStatus(), delete()
4. Write DocumentRepository.ts: getById(), getByProspect(), getByTenant(), create(), markSigned(), delete()
5. Write ActivityRepository.ts (audit log — append only): log(), getByProspect(), getByTenant() This collection is never updated or deleted. It is a permanent audit trail of everything that happens in the system.
6. Write SupplementRepository.ts: getById(), getByTenant(), getPendingCommissions(), create(), update() Commission amount = claimValue × 0.10. Calculate on every create and update that touches claimValue.
7. Write UserRepository.ts: getById(), getByTenant(), getRuferosByTenant(), create(), update(), addFcmToken(), setActive()
8. Write CallLogRepository.ts and SmsLogRepository.ts: create(), getByProspect(), getByAgent(), getByTenant() Both are append-only (no update, no delete).

FILES: functions/src/repositories/TenantRepository.ts,
functions/src/repositories/ProspectRepository.ts,
functions/src/repositories/AppointmentRepository.ts,
functions/src/repositories/DocumentRepository.ts,
functions/src/repositories/ActivityRepository.ts,
functions/src/repositories/SupplementRepository.ts,
functions/src/repositories/UserRepository.ts,
functions/src/repositories/CommunicationRepository.ts, functions/src/repositories/index.ts

AI WORKFLOW: Prompt Claude Code: “Review ProspectRepository.getPage(). It must always include: (1) tenantId filter, (2) doNotCall: false filter, (3) limit of exactly 60 records (not configurable per call), (4) cursor-based pagination using startAfter. Show me any query that could return prospects from a different tenant.”

DONE WHEN: - npm --prefix functions run build still compiles with zero errors - Every repository method includes tenantId in every Firestore query - ProspectRepository.getPage() returns exactly 60 records - No Firestore calls exist outside of repository files (grep -r “db.collection” src/ shows only repository files)

UNBLOCKS: Step 5 (services compose repositories and adapters together)

—

STEP 5 — Backend: Tenant Provisioning Service

WHAT: Write TenantProvisioningService — the most critical function in the system. It creates a fully operational tenant in under 2 minutes.

WHY FIRST: Every other feature depends on tenants existing. Before you can test any other callable function, you need a way to create tenants. This service also establishes the pattern: services orchestrate multiple repositories and adapters to complete a business process.

PERFORMANCE TARGET: The synchronous portion (steps 1-4 below) must return to the caller in under 30 seconds. The full provisioning including Telnyx, SendGrid, and Stripe must complete within 2 minutes. This is validated against the PoC which achieved <2 minute provisioning.

ACTIONS: 1. Write functions/src/services/TenantProvisioningService.ts The provision() method must execute in this exact order:

SYNCHRONOUS (must complete before returning):

Step 1: Create Firebase Auth user (auth.createUser)
Step 2: Create Firestore tenant document (tenantRepository.create)
Step 3: Set Auth custom claims (auth.setCustomUserClaims)
Step 4: Create Firestore user document (userRepository.create)
Step 5: Seed default message templates (batch write)
Return result to caller here.

ASYNCHRONOUS (fire-and-forget, run in parallel after return):

Step 6: provisionTelnyx() – purchase number, create Call Control App
Step 7: provisionSendGrid() – create subuser
Step 8: provisionStripe() – create customer + subscription
Step 9: Send welcome email
Step 10: Log activity

Each async step must have its own catch() handler. A Telnyx failure must NOT roll back or cancel the tenant. It logs the error and a human fixes it manually. Tenant exists regardless.

2. The domain name is generated by slugifying the company name: “Storm Pro Roofing” – “storm-pro-roofing” – storm-pro-roofing.roofaid.app Slugify: lowercase, replace non-alphanumeric with hyphens, trim hyphens.
3. Default calling hours seeded in tenant document: Mon-Sat: 08:00-20:00, Sun: 10:00-18:00
4. Default message templates (3) seeded in message_templates collection: Introduction (SMS), Appointment Confirmation (SMS), Follow Up (SMS)

FILES: functions/src/services/TenantProvisioningService.ts

AI WORKFLOW: After writing, prompt: “Review TenantProvisioningService.provision(). I need to confirm: (1) the function returns to the caller after the synchronous steps, (2) each async step has independent error handling, (3) a failure in step 6, 7, or 8 cannot throw an unhandled exception that crashes the function, (4) the domain is collision-safe (two companies with similar names won’t get the same slug). Show me any issues.”

DONE WHEN: - provisionTenant callable function deployed to emulator - Calling it with valid test data creates: 1 Auth user, 1 tenant document, 1 user document, all with consistent tenantId
- Auth custom claims visible in emulator: {tenantId: "xxx", role: "owner"} - Function returns in under 30 seconds (test with console.time) - Deliberately failing the Telnix step does not fail the function

UNBLOCKS: Steps 6-10 (all callable functions need tenants to exist)

—

STEP 6 — Backend: All Remaining Cloud Functions

WHAT: Write all remaining callable functions, HTTP webhook handlers, and scheduled functions. Export everything from functions/src/index.ts.

ACTIONS: Write callable functions in this priority order:

Priority 1 — Auth and tenant management: getWebRtcCredential, createUser, deactivateUser, registerFcmToken

Priority 2 — Prospect operations (core CRM): getProspects, createProspect, updateProspectStatus, setProspectDNC, importProspects, proximitySearch

Priority 3 — Appointments and documents: createAppointment, updateAppointmentStatus, generateDocument, captureSignature

Priority 4 — Communication: sendSMS

Priority 5 — Supplements and billing: createSupplement, generateSupplement (stub), getBillingPortalUrl

Priority 6 — GDPR: exportTenantData, deleteTenantData

Priority 7 — AI stubs (build the interface, not the intelligence): initiateAICall (stub — returns message "AI calling not yet active") analyzeRoofPhotos (stub — returns empty analysis object)

Write HTTP webhook handlers: telnixWebhook — handles call events + inbound SMS + DNC stop words stripeWebhook — handles subscription updates + payment events inboundEmailWebhook — handles SendGrid inbound parse

Write scheduled functions: sendAppointmentReminders (every 60 minutes) — sends SMS 24h and 2h before processMonthlyCommissions (1st of month, 6am UTC)
generateMonthlyStatement (1st of month, 8am UTC) sendTrialEmailSequence (daily 9am UTC) — Day 7 tips, Day 12 urgency, Day 14 expired + suspend computeDailyMetrics (daily 3am UTC) garbageCollection (daily 2am UTC)

Write functions/src/index.ts exporting all 26 functions by name.

IMPORTANT — Every callable function must: a. Call requireAuth() first — throws if not authenticated b. Verify the user's role is allowed for that operation c. Never query data outside

the authenticated user's tenantId d. Log to ActivityRepository when the action modifies data e. Use secrets via .value() — never read at module load time

FILES: functions/src/functions/callable/index.ts, functions/src/functions/http/index.ts, functions/src/functions/scheduled/index.ts, functions/src/index.ts

AI WORKFLOW: For each callable function, prompt Claude Code: “Review [functionName]. Check: (1) auth verified first, (2) role check before any DB operation, (3) tenantId from token used — never from request.data, (4) activity logged after successful write, (5) error handling returns typed HttpsError not generic Error.”

For webhook handlers, prompt: “Review telnyxWebhook. It must always return HTTP 200 to Telnyx — even on internal errors — to prevent Telnyx from retrying. Show me any code path that could return non-200.”

DONE WHEN: - npm -prefix functions run build compiles with zero errors - firebase emulators:start shows all functions loading - provisionTenant callable creates a complete tenant - getProspects callable returns 60 records for valid tenant - telnyxWebhook HTTP endpoint returns 200 for all event types - stripeWebhook verifies signature and processes subscription.updated

UNBLOCKS: Step 7 (frontend can now call these functions)

—

STEP 7 — Backend: PDF Generation Service

WHAT: Write the PDF generation service that produces third-party authorization forms, ACV contracts, and RCV contracts. These PDFs must be professional and field-ready — a homeowner will sign them.

ACTIONS: 1. Write functions/src/services/PdfGenerationService.ts using pdf-lib. generateDocument() — creates a PDF from template type + data embedSignature() — loads existing PDF, embeds signature PNG, adds signer name and date, saves as new signed version

2. Document templates to implement: third_party_auth — 3rd Party Authorization form acv_contract — Roofing Contract (Actual Cash Value) rcv_contract — Roofing Contract (Replacement Cost Value)
3. All PDFs must include: Orange header bar (#E8501F) with company name Homeowner info block (name, address, city, state, zip, phone) Body text specific to document type Signature line at bottom with date field “Electronically signed via Roof-Aid CRM” footer text
4. generateDocument() uploads PDF to Cloud Storage via StorageAdapter and returns the storage path.
5. embedSignature() accepts a base64-encoded PNG of the signature, loads the original PDF from Storage, draws the signature image onto the last page, adds typed name and date, saves the signed version to a new path, and returns that path.

FILES: functions/src/services/PdfGenerationService.ts

AI WORKFLOW: Prompt Claude Code: “Review the signature embedding in PdfGenerationService.embedSignature(). The signed PDF must: (1) place the signature image on the correct line, not over the text, (2) add a timestamp in the format ‘Signed: [date] via Roof-Aid CRM’, (3) be a valid PDF that can be opened in iOS Files and Adobe Acrobat. Check for any pdf-lib API calls that changed between v1.16 and v1.17.”

DONE WHEN: - generateDocument() produces a readable PDF with correct formatting - embedSignature() produces a PDF with visible signature, name, and date - Both PDFs open correctly in macOS Preview, Adobe Acrobat, and iOS Files app - Files upload to Cloud Storage emulator at the expected paths

UNBLOCKS: Step 8 (document module in the frontend uses this service)

STEP 8 — Deploy Backend to Staging

WHAT: Deploy the complete backend to the roof-aid-dev Firebase project. This is the first real deployment. Smoke test every function.

ACTIONS: 1. Set secrets in the dev project (use real test API credentials): firebase secrets:set TELNYX_API_KEY –project roof-aid-dev firebase secrets:set SENDGRID_API_KEY –project roof-aid-dev (repeat for all 6 secrets)

2. Deploy all functions: firebase deploy –only functions –project roof-aid-dev
3. Smoke test each critical function using the Firebase console or a simple test script:
 - provisionTenant: creates a real tenant with real Telnix number
 - getWebRtcCredential: returns a valid token
 - getProspects: returns empty array for new tenant (not an error)
 - telnyxWebhook: POST with test event body returns 200
 - stripeWebhook: POST with test event returns 200
4. Check Cloud Functions logs for any cold start errors. Target: cold start under 2.1s (matching PoC baseline).
5. Verify all 23 Firestore indexes are READY in the dev project.

FILES: No new files. This step is deployment and testing only.

AI WORKFLOW: Prompt Claude: “I deployed my Cloud Functions and I’m seeing this error in the logs: [paste error]. What is the likely cause and fix?” Use this for any deployment errors. Do not guess at fixes for Cloud Functions errors — they are often environment-specific.

DONE WHEN: - All functions deployed with zero deployment errors - provisionTenant creates a real tenant end-to-end in dev - No function throws an unhandled exception in the first 5 calls -

Cold start time under 2.5s (slightly above baseline is acceptable for the first deploy — optimize later)

UNBLOCKS: Step 9 (frontend now has real functions to call)

————— PHASE 2 — FRONTEND (WEB APP) Goal: A complete, production-ready React web app with all 10 screens, PWA support, and real-time data. Target: 3 weeks.

STEP 9 — Frontend: Project Setup, Firebase Config, Auth

WHAT: Initialize the React + Vite frontend, connect to Firebase v10, implement login/logout with role-based routing, and register the service worker.

PERFORMANCE TARGET: Match the PoC baseline — FCP under 1.2s on 4G, TTI under 2.8s on 4G, bundle size under 650KB gzipped (PoC was 580KB; React adds ~45KB, Zustand adds ~15KB, leaving 10KB headroom).

ACTIONS: 1. Initialize frontend with Vite + React + TypeScript: `npm create vite@latest frontend --template react-ts cd frontend && npm install`

2. Install dependencies: `react-router-dom@6, zustand@4, firebase@10, date-fns@3, react-hot-toast@2, lucide-react, @googlemaps/js-api-loader`
3. Write `frontend/src/config/firebase.ts` using Firebase v10 modular API: `initializeApp()` — singleton pattern `initializeAppCheck()` with `ReCaptchaV3Provider` — REQUIRED, not optional `getAuth()` with `setPersistence(browserLocalPersistence)` `getFirestore()` with `enableIndexedDbPersistence()` `getFunctions()`, `getStorage()`, `getMessaging()` (check `isSupported` first) IMPORTANT: Use `browserLocalPersistence` constant (v10 API). Do NOT use `firebase.auth.Auth.Persistence.LOCAL` (v8 API).
4. Write `frontend/src/stores/authStore.ts` (Zustand): State: `user (AppUser | null)`, `firebaseUser`, `loading`, `error` Actions: `init()` — sets up `onAuthStateChanged` listener, returns `unsubscribe` `login()`, `logout()`, `resetPassword()`, `clearError()` The `init()` action forces a token refresh to get latest custom claims. The `AppUser` type includes: `uid`, `email`, `name`, `role`, `tenantId`, `telnyxNumber`
5. Write `frontend/src/main.tsx`: `BrowserRouter` with nested routes `RequireAuth` wrapper — redirects to `/login` if not authenticated `RequireRole` wrapper — redirects to `/dashboard` if wrong role Register service worker in production only `Toaster` component (`react-hot-toast`) for global notifications
6. Write `frontend/public/manifest.json` (PWA): `name`, `short_name`, `start_url`, `display`: `standalone`, `background_color`: `#0F172A`, `theme_color`: `#E8501F`, 8 icon sizes (72, 96, 128, 144, 152, 192, 384, 512)
7. Write `frontend/public/service-worker.js`: Install event: `precache` critical assets Activate event: `delete` old caches Fetch event: `cache-first` for static assets, `network-first` for

navigation Sync event: process offline queue (IndexedDB-based write queue) Push event: show FCM notifications

8. Write frontend/src/pages/LoginPage.tsx: Email + password form, show/hide password toggle Calls authStore.login(), redirects to /dashboard on success Orange brand color (#E8501F) for the submit button Roof-Aid logo mark (R in orange square) as header
9. Write frontend/src/components/layout/AppLayout.tsx: Left sidebar (collapsible to icon-only mode) Navigation links filtered by user role Header with: online/offline indicator, notification bell with badge, user avatar, logout button for page content Sidebar brand color: #1E293B (dark slate), accent: #E8501F

FILES: frontend/package.json, frontend/vite.config.ts, frontend/index.html, frontend/public/manifest.json, frontend/public/service-worker.js, frontend/src/config/firebase.ts, frontend/src/stores/authStore.ts, frontend/src/main.tsx, frontend/src/pages/LoginPage.tsx, frontend/src/components/layout/AppLayout.tsx

AI WORKFLOW: After writing firebase.ts, prompt Claude Code: “Audit this Firebase v10 initialization. Flag any v8/v9 compat API calls that were changed in v10, especially around Auth persistence, Firestore listeners, and Storage download URLs. The project must not use the compat layer.”

After writing the service worker, prompt: “Review this service worker. It must: (1) not intercept Firebase/Google API calls, (2) not break the Firestore real-time listener WebSocket connection, (3) handle the case where the user is offline and tries to navigate to a new route.”

DONE WHEN: - npm run dev starts with zero errors - Login page renders and accepts credentials - Successful login redirects to /dashboard - Logout clears the session and redirects to /login - A user with role=telefonista cannot navigate to /admin (redirected) - Lighthouse PWA score shows installable: YES - Bundle size reported by vite build under 650KB gzipped

UNBLOCKS: Step 10 (dashboard and all feature pages)

—

STEP 10 — Frontend: Zustand Stores and Data Layer

WHAT: Write all frontend stores before building any page components. Stores manage real-time data subscriptions and state. Pages just render what stores provide.

ACTIONS: 1. Write frontend/src/stores/prospectStore.ts: State: prospects[], displayList[] (anti-collision rotated), selected, filters, loading, error, lastDoc (for pagination cursor) Actions: fetchProspects(), loadMore(), selectProspect(), updateLocalStatus() (immediate local update), removeFromList() (removes after status change)

CRITICAL — Anti-Collision rotation must be implemented here:

After fetching 60 records from the backend, apply this rotation:

```
const offset = new Date().getSeconds(); // 0-59
const clamped = Math.min(offset, prospects.length - 1);
displayList = [...prospects.slice(clamped), ...prospects.slice(0,
```

clamped)];

This is what makes the list order different for each agent.

Display the `displayList`, not the raw prospects array.

2. Write `frontend/src/stores/notificationStore.ts`: Real-time Firestore listener on notifications collection. State: `notifications[]`, `unreadCount` Actions: `startListening(userId)` → returns `unsubscribe`, `markRead(id)`, `markAllRead()`
3. Write `frontend/src/stores/appointmentStore.ts`: Fetches appointments for the current tenant within a date range. State: `appointments[]`, `selected`, `loading` Actions: `fetch(from, to)`, `create()`, `updateStatus()`
4. Write `frontend/src/stores/smsStore.ts`: Real-time listener on `sms_logs` filtered by `tenantId` and `prospectId`. State: `threads (Map<prospectId, SmsMessage[]>)`, `unreadCount` Actions: `startListening()`, `send()`, `markRead()`
5. Write `frontend/src/stores/analyticsStore.ts`: Fetches `daily_metrics` for the current tenant. State: `metrics[]`, `summary`, `loading` Actions: `fetch(dateRange)`

FILES: `frontend/src/stores/prospectStore.ts`, `frontend/src/stores/notificationStore.ts`, `frontend/src/stores/appointmentStore.ts`, `frontend/src/stores/smsStore.ts`, `frontend/src/stores/analyticsStore.ts`

AI WORKFLOW: Prompt Claude Code: “Review `prospectStore.ts`. Specifically check the Anti-Collision rotation logic. The rotation must produce a different order each second. Two agents loading at `second=5` must see the same order, but an agent loading at `second=5` and one at `second=10` must see different orders. Show me a test that verifies this.”

DONE WHEN: - All stores compile with zero TypeScript errors - `prospectStore.displayList` is always length 60 or less - `prospectStore.displayList` order changes every second (verify by calling `fetchProspects()` twice with a 2-second gap) - `notificationStore` shows real-time updates when a notification is written directly to Firestore emulator

UNBLOCKS: Steps 11-14 (all pages consume these stores)

—

STEP 11 — Frontend: Dashboard Page

WHAT: The main screen every agent sees on login. Map + prospect list with the Anti-Collision rotated order, city and status filters, proximity search, and 6-button quick action cards.

ACTIONS: 1. Write `frontend/src/pages/DashboardPage.tsx`: Left panel (40%): Filters bar (city dropdown, status filter), Prospect card list (from `prospectStore.displayList` — 60 records), “Load 60 More” button for next page Right panel (60%): Google Maps with prospect pins, color-coded by status

2. Prospect card layout (each of the 60 cards): Homeowner name (bold), address line, city Home value badge if available, hail size badge if available Status pill (color-coded) 6

action buttons: Call | SMS | Email | Appt | Go | Notes DNC is NOT a card button. It appears on the full profile only.

3. Google Maps integration: Load map using @googlemaps/js-api-loader with the Maps API key from the environment variable Plot all 60 prospects as pins (color-coded by status) Click a pin: opens the prospect card in a side panel Right-click on any map point: opens Proximity Search modal (pre-fills lat/lng from the click position) Proximity search button in top bar: uses current GPS location
4. Proximity Search modal: Center point (lat/lng — auto-populated from click or GPS), Radius selector: 5 / 10 / 25 / 50 miles, Status filter (optional), Search button → calls proximitySearch Cloud Function
5. Record count display: “60 de 180” format (loaded of total)
6. Write frontend/src/components/ui/OnboardingChecklist.tsx: Shown to Owner role only, on first login, as a right-side panel. 7 steps: Add team members, Import leads, Make first call, Set up calling hours, Add a Rufero, Set service area, Download mobile app. Progress: “3 of 7 complete”. Each step has an action button. Dismissible only after all 7 are complete.

FILES: frontend/src/pages/DashboardPage.tsx,
frontend/src/components/ui/OnboardingChecklist.tsx

AI WORKFLOW: Prompt Claude Code: “Review DashboardPage. The prospect list displays prospectStore.displayList — the anti-collision rotated array. When a user clicks ‘Call’ on a card, it must: (1) immediately call updateLocalStatus(id, ‘contacted’) to remove the prospect from the visible top of the list, (2) open the softphone, (3) the list reorders based on the new rotation. Verify this flow is correct.”

DONE WHEN: - Dashboard renders with 60 prospect cards from the store - Map renders with color-coded pins for each prospect - Right-click on map opens Proximity Search modal with lat/lng pre-filled - Clicking Call on a card removes it from the top of the list immediately - Onboarding checklist appears for Owner role on first login - Onboarding checklist does NOT appear for Telefonista role

UNBLOCKS: Step 12 (prospect profile is linked from dashboard cards)

—

STEP 12 — Frontend: Remaining Pages (All 8)

WHAT: Build all remaining pages. They share the same patterns — load from stores, call Cloud Functions for mutations, show toast notifications for feedback.

Build in this order:

12A — Prospect Profile Page (/prospects/:id) Tabs: Overview, Timeline, Calls, SMS, Appointments, Documents Overview tab: full prospect details, edit inline, DNC button Timeline

tab: activity log in chronological order Each tab loads lazily (only when clicked) Back button returns to dashboard with scroll position preserved

12B — Softphone Page (/phone) Telnyx WebRTC integration (same library as PoC: ~180KB) Dial pad, active call controls (mute, hold, hang up, transfer) Call history sidebar (recent calls with duration and recording link) Integration: clicking Call from dashboard pre-populates the number Recording disclosure shown before call is placed

12C — SMS Page (/sms) Left panel: conversation list (all prospects with SMS threads) Right panel: threaded message view for selected prospect Message input with send button and template picker Real-time updates via smsStore Firestore listener Unread count badge in sidebar navigation

12D — Appointments Page (/appointments) Calendar view (month/week/day) showing all appointments Color-coded by status (pending=orange, confirmed=green, etc.) Click a day: opens appointment creation form Click an appointment: opens appointment detail with status buttons Rufero filter: filter calendar by assigned Rufero

12E — Documents Page (/documents) List of all generated documents for current tenant Filter by type and status Generate New Document button: opens form (select prospect + type) Each row: document type, prospect name, status, date, action buttons Signed documents show a green checkmark and download button

12F — Supplements Page (/supplements) List of all supplement claims Create New Supplement button: line item editor form Each supplement: claim value, commission amount (10%), status Commission status badge (pending/billed/paid) AI Generate button (shows stub message until model is connected)

12G — Analytics Page (/analytics) KPI cards: total prospects, appointments set, contracts signed, conversion rate (lead → appointment), conversion rate (appt → contract) Line chart: daily call volume (last 30 days) Bar chart: appointments by Rufero Pipeline funnel: prospects by status Date range filter

12H — Admin Page (/admin) — Owner and Admin roles only Tabs: Users, Import, Settings, Activity Log Users tab: list of team members, add/deactivate buttons Import tab: CSV upload for prospect bulk import Settings tab: calling hours, recording disclosure, service area Activity Log tab: recent activities across the whole tenant

12I — Billing Page (/billing) — Owner role only Current plan and status (trial/active/suspended) Trial days remaining (if in trial) Stripe billing portal button (calls getBillingPortalUrl) Invoice history (last 12 invoices) Upgrade/downgrade plan buttons

12J — Super Admin Page (/superadmin) — super_admin role only Tenant list with status, tier, and created date Provision new tenant form Feature flag overrides per tenant Platform-wide metrics

FILES: frontend/src/pages/ProspectPage.tsx, frontend/src/pages/SoftphonePage.tsx, frontend/src/pages/SMSPage.tsx, frontend/src/pages/AppointmentsPage.tsx, frontend/src/pages/DocumentsPage.tsx, frontend/src/pages/SupplementsPage.tsx,

frontend/src/pages/AnalyticsPage.tsx, frontend/src/pages/AdminPage.tsx,
frontend/src/pages/BillingPage.tsx, frontend/src/pages/SuperAdminPage.tsx

AI WORKFLOW: Build one page at a time. For each page, start with this prompt: “I’m building [PageName] for a roofing CRM. The user role is [role]. The data comes from [storeName]. The mutations go through Cloud Functions. Generate the complete page component. After generating, flag any: (1) direct Firestore calls that should go through the store, (2) missing loading states, (3) missing error states, (4) actions that don’t show a toast notification.”

DONE WHEN: - All 10 pages render without console errors - Every data mutation shows a toast (success or error) - Every page has a loading state (spinner or skeleton) - Every page has an error state (error message with retry) - Role-restricted pages redirect correctly (e.g. /admin for telefonista)

UNBLOCKS: Step 13 (AI support widget added to all pages)

STEP 13 — Frontend: AI Support Widget

WHAT: A chat widget in the bottom-right corner of every page that connects to Claude and answers questions about the CRM.

ACTIONS: 1. Write frontend/src/components/ui/AISupportWidget.tsx: Floating button (bottom-right, orange circle with chat icon) Click: opens chat panel (320px wide, 480px tall) Chat panel has: message history, input field, send button First message pre-populated: “Hi! I’m your Roof-Aid assistant. Ask me anything about the CRM or your account.”

2. Connect to Claude API (claude-sonnet-4-20250514): POST to /v1/messages with system prompt: “You are the support assistant for Roof-Aid CRM, a platform for roofing companies. Help users with: navigating the CRM, understanding their pipeline, using the softphone, creating appointments, uploading documents, and understanding their billing. Be concise. The user’s role is [role].” Include the user’s role in the system prompt dynamically.
3. API key: stored in Firestore under platform_config/support Fetched via a Cloud Function — never exposed to the frontend. Write a getSupportToken Cloud Function that returns a short-lived token for the support chat.
4. The widget must appear on every page — it is rendered in AppLayout.tsx, not inside individual pages.
5. If the Claude API call fails, show: “Support chat is temporarily unavailable. Email support@roofaid.app for help.”

FILES: frontend/src/components/ui/AISupportWidget.tsx,
functions/src/functions/callable/getSupportToken.ts

AI WORKFLOW: This is a meta step — you are using Claude to build a Claude-powered widget. Prompt: “I need a React chat widget that calls the Anthropic Messages API. It must: (1)

maintain conversation history in local state (not persisted), (2) show a typing indicator while waiting, (3) handle API errors gracefully, (4) never expose the API key in the frontend bundle. The API key is fetched from a Cloud Function.”

DONE WHEN: - Widget appears on all 10 pages in the bottom-right corner - Sending a question returns a relevant answer from Claude - Widget does not appear on the Login page - Closing the widget preserves the conversation history until page refresh

UNBLOCKS: Step 14 (frontend deploy to staging)

STEP 14 — Deploy Frontend to Staging and Full Integration Test

WHAT: Build and deploy the frontend to Firebase Hosting (dev project). Run an end-to-end test covering the full workflow from lead to signed contract.

ACTIONS: 1. npm --prefix frontend run build Verify bundle size: vite build outputs gzip sizes in the build log. Target: under 650KB gzipped. If over, identify which chunks are large and add them to the manualChunks config in vite.config.ts.

2. firebase deploy --only hosting --project roof-aid-dev
3. Run this end-to-end test scenario manually:
 - a. Super admin provisions a new test tenant (provisionTenant)
 - b. Owner logs in — onboarding checklist appears
 - c. Owner creates 3 team members (1 admin, 1 telefonista, 1 rufero)
 - d. Admin imports 10 test prospects via CSV
 - e. Telefonista logs in — dashboard shows rotated list of prospects
 - f. Telefonista clicks Call on a prospect — softphone opens
 - g. After “call”, telefonista marks prospect as Contacted
 - h. Prospect disappears from top of list
 - i. Telefonista creates an appointment for the prospect
 - j. Prospect status updates to Scheduled
 - k. Rufero logs in on mobile — sees appointment in schedule
 - l. Rufero generates a 3rd Party Authorization document
 - m. Homeowner signs the document
 - n. Prospect status updates to Closed
 - o. Supplement is created for the closed prospect
 - p. Admin views the analytics page — all metrics reflect activity
4. Check Firebase console for any unhandled errors in Functions logs.

DONE WHEN: - All 16 steps of the end-to-end test complete without errors - Bundle size under 650KB gzipped - Lighthouse score 85+ on the deployed staging URL - No console errors in browser devtools during the full workflow - Firebase Functions log shows zero unhandled exceptions

UNBLOCKS: Phase 3 (mobile app)

————— PHASE 3 — MOBILE APP (React Native) Goal: iOS and Android apps covering the Rufero workflow — schedule, navigate, inspect, sign. Target: 3 weeks.

STEP 15 — Mobile: Setup and Core Screens

WHAT: Initialize the React Native project, connect to Firebase, and build the 7 core screens for the Rufero role.

ACTIONS: 1. Initialize with Expo (managed workflow): `npx create-expo-app mobile --template expo-template-blank-typescript` Install: `@react-native-firebase/app`, `@react-native-firebase/auth`, `@react-native-firebase/firestore`, `@react-navigation/native`, `@react-navigation/bottom-tabs`, `expo-camera`, `expo-location`, `expo-local-authentication` (biometrics), `expo-notifications`

2. Firebase config: same project as web, different app registration. Download `google-services.json` (Android) and `GoogleService-Info.plist` (iOS).
3. App Check for mobile: DeviceCheck provider for iOS (requires Apple Developer account). Play Integrity provider for Android. This is required before App Store submission.
4. Build these 7 screens in this order: LoginScreen — email/password + biometric toggle (Face ID / fingerprint) ScheduleScreen — list of today's appointments, tap to view detail AppointmentDetailScreen — full details, status buttons (Confirm/Complete/ No-show/Cancel), Navigate button (opens Maps app), Call and SMS buttons ProspectDetailScreen — read-only view of prospect info, call history InspectionScreen — photo capture (up to 20 photos), damage notes form, roof measurements, signature capture pad DocumentViewerScreen — PDF viewer for signed documents (pinch to zoom), share button (email or native share sheet), download button SMSInboxScreen — threaded SMS for assigned prospects, reply from app, unread count badge on tab bar SettingsScreen — profile photo, notification preferences, biometric toggle, app version, logout
5. Bottom tab navigation: Schedule | Prospects | Documents | SMS | Settings
6. Offline mode: All appointment data cached to device on login. Status updates queued when offline, synced when connection restores. Show offline banner when no network.

FILES: `mobile/app.json`, `mobile/App.tsx`, `mobile/src/screens/.tsx`, `mobile/src/navigation/AppNavigator.tsx`, `mobile/src/stores/.ts` (mirror relevant web stores)

AI WORKFLOW: Prompt Claude Code: "I'm building InspectionScreen in React Native with Expo. It needs: (1) camera roll access to capture up to 20 photos, (2) a signature pad using expo-signature (or hand-drawn with a canvas), (3) offline capability — all photos saved to device, uploaded to Firebase Storage when online. Show me the complete screen with proper permissions handling for both iOS and Android."

DONE WHEN: - App runs in Expo Go on a physical iOS and Android device - Login with biometrics works on devices that support it - Appointment detail screen shows real data from Firestore - Photo capture saves images and uploads to Cloud Storage - Signature capture produces a PNG that the web app can use - Offline mode queues a status update and syncs on reconnect

UNBLOCKS: Step 16 (App Store and Play Store submission)

—

STEP 16 — Mobile: App Store and Play Store Submission

WHAT: Build production binaries and submit to both stores.

ACTIONS: 1. Create app icons (all required sizes) and splash screen. 2. Configure app.json: name, slug, version, bundle identifiers. 3. Apple Developer account: create App ID, provisioning profile, certificates. 4. Google Play Console: create app, upload signed APK. 5. Build with EAS Build: eas build --platform all 6. Submit for review. Apple review: 1-3 days. Google: 1-3 days. 7. Note app store URLs for marketing website and onboarding checklist.

DONE WHEN: - App approved and live in App Store (iOS) - App approved and live in Google Play Store (Android) - App Store URL added to onboarding checklist Step 7

————— PHASE 4 — STORM LEADS MODULE (Tier 2) Goal: Automated storm lead import when a hail event hits a tenant's service area. This unlocks the Tier 2 subscription.

STEP 17 — Storm Leads

WHAT: Integrate a storm data provider, configure service area per tenant, and automatically import prospects when a hail event occurs.

ACTIONS: 1. Select and integrate storm data provider (StormAware, HailTrace, or equivalent). Sign up for API access. Store API key in Secret Manager.

2. Write a scheduled Cloud Function (stormLeadWatcher) that runs daily:
 - a. For each active Tier 2+ tenant, fetch their service area cities
 - b. Query the storm data API for hail events in the last 24 hours matching the tenant's service area
 - c. For each new hail event, fetch affected addresses from the provider
 - d. Call prospectRepository.bulkCreate() with the new leads
 - e. Send a push notification to the tenant's owner: "New storm leads available: 47 properties affected in [city]"
 - f. Log the import to ActivityRepository

3. Add storm radar overlay to the dashboard map (Tier 2+ only): Fetch radar data from a weather API (OpenWeatherMap or equivalent) Overlay as a semi-transparent layer on the Google Map Toggle button in the map controls
4. Service area configuration in Admin — Settings: Cities list (add/remove), center point on map, radius in miles
5. Tier 2 feature flag gates this entire module. `tenantRepository.hasFeature(tenantId, 'leads')` must return true. If Tier 1 user navigates to storm features, show upgrade prompt.

DONE WHEN: - stormLeadWatcher runs without error and imports test leads - Storm radar overlay toggles on the dashboard map for Tier 2 tenants - Tier 1 tenants see an upgrade prompt instead of storm features - New leads appear in the dashboard within 2 minutes of the scheduled function running

————— PHASE 5 — AI CALLING AGENT (Tier 3) Goal: Replace or augment the human telefonista with an AI agent that calls prospects, qualifies them, and books appointments automatically.

STEP 18 — AI Calling Agent

WHAT: Connect the `initiateAICall` stub to a real LLM + Telnyx Call Control bridge. The AI speaks with homeowners, qualifies them, and books appointments.

IMPORTANT: This is the most complex module. Do not start it until Phases 1-4 are fully complete and in production with real paying customers. The stub handles graceful degradation until this phase.

ACTIONS: 1. Design the conversation flow: Opening: “Hi, is this [name]? This is [company] calling about your roof. We’re reaching out because a hailstorm recently hit your area. We offer FREE roof inspections. Do you have a moment?” Qualification questions: Do they own the home? Has the roof been inspected recently? Do they have homeowner’s insurance? Appointment booking: “Would [day] at [time] work for a FREE inspection?” Objection handling: 3 scripted responses per common objection. Voicemail: leave a 20-second message and hang up.

2. Technical implementation: Telnyx Call Control: initiates the call, streams audio to/from the AI Speech-to-text: transcribe homeowner’s voice in real time LLM (Claude claude-sonnet-4-20250514): process transcript, decide next line Text-to-speech: convert AI response to audio, play via Telnyx Latency target: under 1 second from homeowner speaking to AI responding
3. Human handoff: when the AI detects high intent or confusion, it says “Let me connect you with one of our team members” and transfers the call to the next available telefonista.

4. After each call: store transcript, outcome (qualified/not/voicemail), and appointment details if booked.
5. Tier 3 feature flag gates this entire module.

DONE WHEN: - AI completes a test call with a real phone number end-to-end - AI successfully books an appointment and creates it in Firestore - Human handoff transfer completes without call drop - Transcripts stored for every call

————— PHASE 6 — SUPPLEMENTS (Tier 4) Goal: Line-item supplement editor, commission calculation, automated monthly billing to clients, and ML-assisted line item suggestion.

STEP 19 — Supplements and Commission Billing

WHAT: The Supplements module is already partially built (the UI page, the repository, the monthly billing function). This phase connects all the pieces and adds the ML-assisted generation.

ACTIONS: 1. Complete the Supplements page UI: Line item editor: description, quantity, unit, unit price, total Auto-calculated commission display: $\text{claimValue} \times 10\% = \text{commission}$ Status workflow: Draft → Submitted → Accepted/Denied Submit to insurance button (generates a PDF of the supplement)

2. Commission billing is already implemented in `processMonthlyCommissions` (scheduled function). Verify it runs correctly with real supplement data.
3. Commission statement PDF: `generateMonthlyStatement` is already implemented. Verify the email is formatted correctly.
4. ML supplement generation: replace the `generateSupplement` stub. Options: train a model on historical supplement data, or use Claude to suggest line items based on inspection photos and damage description. Start with Claude (faster to launch). Prompt: “Based on this roof inspection: [damage description], [hail size: X inches], [roof age: Y years], [roof type: Z], suggest insurance supplement line items with typical pricing.”
5. Tier 4 feature flag gates this module. `supplementCommission` flag is separate from `supplements` flag. A tenant can have the writing tool without auto-billing.

DONE WHEN: - Supplement created, submitted, and marked accepted → triggers commission - Monthly billing function bills the correct amount - Monthly statement PDF emailed to tenant billing address - ML generation returns at least 5 suggested line items for a test inspection

————— PHASE 7 — AI ROOF INSPECTION (Tier 5) Goal: Computer vision analysis of roof photos to identify damage, estimate severity, and auto-populate supplement line items.

STEP 20 — Computer Vision Module

WHAT: Replace the analyzeRoofPhotos stub with a real computer vision model. This is the highest-value feature for insurance claim processing.

ACTIONS: 1. Model options: Google Cloud Vision API (fastest to integrate), AWS Rekognition, or a custom model fine-tuned on roof damage photos. Recommended starting point: Google Cloud Vision API + Claude vision to interpret the results in roofing industry terms.

2. analyzeRoofPhotos callable function implementation: Accept up to 20 photos (base64 or Storage paths) For each photo: detect damage type, affected area percentage, severity Aggregate: overall condition score (1-10), primary damage type, estimated age, recommended action (repair vs replace) Return: damageScore, affectedAreas[], estimatedDamage\$, confidence%
3. CV findings automatically populate supplement line items: If CV detects “hail impact on field shingles, 60% coverage” → suggest line item: “Replace field shingles — [sq footage]”
4. AI Analysis card in mobile app (InspectionScreen): After uploading photos, show “Analyze with AI” button Display results as a visual damage report card
5. Automated damage report PDF: generateDocument() with new type ‘cv_damage_report’ — includes photo thumbnails and AI findings.
6. Tier 5 feature flag gates this module.

———— PHASE 8 — MARKETING WEBSITE AND ACQUISITION Goal: A public-facing website that explains the product, shows pricing, and converts visitors to free trial signups.

STEP 21 — Marketing Website

ACTIONS: 1. Build at roofaid.app (or chosen domain) as a separate Vite site or using a CMS (Webflow, Framer, or similar for speed).

2. Required pages: Home, Pricing, Features, About, Blog (optional), Free Trial signup, Login redirect.
3. Pricing page must match the tier structure in this spec exactly. All prices pulled from the platform_config Firestore document (same source of truth as the app). Never hardcoded.
4. Free Trial form calls provisionTenant Cloud Function directly. On success: redirect to the new tenant’s URL (tenant.domain/login) with a “Your account is ready” message.

5. SEO: meta tags, structured data (SoftwareApplication schema), sitemap.xml, robots.txt.
6. Analytics: add Google Analytics or Plausible for conversion tracking.

————— AFTER EACH PHASE — STANDARD CHECKLIST

After completing each phase, before starting the next:

SECURITY: ☐ Run: firebase deploy –only firestore:rules Verify rules still block cross-tenant access ☐ Check Cloud Functions logs for any permission denied errors that suggest a security rule gap

PERFORMANCE: ☐ Run Lighthouse on the staging URL. Score must be 85+. ☐ Measure cold start time. Must be under 2.5s (target: match the PoC baseline of 2.1s p95). ☐ Check bundle size. Must stay under 650KB gzipped.

DATA: ☐ Verify all new Firestore indexes are in READY state ☐ Spot-check that new documents include tenantId field

AI REVIEW: ☐ Paste the new code into Claude and ask: “What are the top 3 security risks in this code? What are the top 3 performance issues?” ☐ Fix any critical findings before deploying to production.

DEPLOYMENT: ☐ Deploy to roof-aid-dev first. Test for 24 hours. ☐ If no issues, deploy to roof-aid-prod. ☐ Never deploy directly to prod without staging validation.

27. Scaling Strategy — Firebase to Cloud-Agnostic Architecture

This section defines the long-term architectural strategy for scaling the platform from launch through hundreds of tenants and beyond. The decisions made on day one determine whether future migration is a surgical operation or a full rewrite. Getting this right is not optional.

CRITICAL FOR DEVELOPER: Every architectural decision in this section must be implemented from the first line of code. Retrofitting these patterns later costs 3-5x more than building them correctly upfront.

27.1 The Core Problem: Tight Coupling

Most early SaaS platforms are built with direct calls to the underlying infrastructure scattered throughout the codebase — Firestore queries written in UI components, Firebase Auth checks mixed into business logic, Cloud Storage paths hardcoded in feature modules. This is fast initially but creates a system impossible to change without touching hundreds of files.

The result: at 300 tenants when Firestore costs hurt margins, the developer says ‘migrating to PostgreSQL is a 6-month rewrite.’ That is a business crisis. The solution is to never allow tight coupling in the first place.

27.2 The Three Required Architectural Patterns

Pattern 1: Repository Pattern (Data Access Layer)

All database operations go through Repository classes. Business logic and UI never make direct database calls. Repositories are the only layer that knows which database is being used.

WRONG — direct Firestore in business logic:

`db.collection('prospects').where('tenantId','==',id).get()` CORRECT — business logic calls repository: `ProspectRepository.getByTenant(tenantId)` To migrate to PostgreSQL later: rewrite only the repository file. Zero changes anywhere else.

- Every Firestore collection has a corresponding Repository class in a /repositories directory
- Repository methods named by business intent: `getByTenant()`, `findByStatus()`, `assignToRufero()`
- Repositories accept and return plain JavaScript objects — never Firestore DocumentSnapshot objects
- No Firestore SDK imports outside of repository files and the firebase config initializer

Pattern 2: Service Adapters (Third-Party Services)

Every external service is wrapped in an Adapter class. The application calls the adapter interface — never the third-party SDK directly.

Service	Adapter	Key Methods
Firebase Auth	AuthAdapter	<code>createUser()</code> , <code>signIn()</code> , <code>setCustomClaims()</code> , <code>revokeSession()</code>
Cloud Storage	StorageAdapter	<code>upload()</code> , <code>getSignedUrl()</code> , <code>delete()</code>
Telnyx	PhoneAdapter	<code>initiateCall()</code> , <code>sendSMS()</code> , <code>getCallStatus()</code>
SendGrid	EmailAdapter	<code>sendEmail()</code> , <code>syncInbox()</code> , <code>createSubuser()</code>
Stripe	BillingAdapter	<code>createSubscription()</code> , <code>createInvoice()</code> , <code>chargeCustomer()</code>
FCM	NotificationAdapter	<code>sendPush()</code> , <code>sendWebPush()</code>
Lead Provider	LeadProviderAdapter	<code>getStormEvents()</code> , <code>getAffectedProperties()</code>

- Adapter classes live in /adapters directory — no third-party SDK imports anywhere else
- Swapping any external service requires rewriting exactly one adapter file — zero business logic changes

Pattern 3: Environment-Based Configuration

- All service credentials and config in environment variables — never hardcoded
- All secrets in Google Secret Manager — fetched at runtime
- A single `config.js` reads all env vars and exports them — app imports from config, not `process.env` directly

- Feature flags stored in Firestore platform_config — changeable without redeployment

27.3 Three-Stage Scaling Strategy

STAGE 1 — Firebase Foundation (Launch to ~200 Tenants)

Layer	Service	Rationale
Database	Cloud Firestore	Zero setup, real-time native, no DBA needed
Auth	Firebase Auth	JWT, custom claims, free tier generous
Storage	Firebase Cloud Storage	Integrated rules, CDN built-in
Backend	Cloud Functions Gen2	Serverless, zero DevOps, auto-scales
Hosting	Firebase Hosting	Global CDN, automatic SSL

Tenants	Est. Firebase Cost/mo	Est. Revenue (avg \$199/tenant)	Gross Margin
10	~\$200	~\$1,990	~90%
50	~\$600	~\$9,950	~94%
100	~\$1,200	~\$19,900	~94%
200	~\$2,400	~\$39,800	~94%

- Firebase is right at this stage — fastest to build, lowest complexity, excellent margins
- All three patterns (Repository, Adapter, Config) built from day one — this is the investment that protects the future
- Exit trigger: Firestore costs exceed 15% of monthly revenue OR complex SQL queries become necessary

STAGE 2 — Hybrid Architecture (~200 to ~500 Tenants)

Keep Firebase for what it does best (real-time, auth, mobile sync). Introduce PostgreSQL for what Firebase does poorly (analytics, reporting, financial data, complex queries). Because Repository Pattern was used, this migration is file-by-file — not a rewrite.

Layer	Stage 1	Stage 2	Change
Primary DB	Firestore	Firestore + PostgreSQL	Analytics/financial repos rewritten to use PostgreSQL
Real-time	Firestore listeners	Firestore listeners	No change — Firestore stays for real-time
Auth	Firebase Auth	Firebase Auth	No change
Analytics	Firestore	PostgreSQL	Analytics repos rewritten — zero

	queries	(SQL)	business logic change
Financial Data	Firestore	PostgreSQL	Supplement commissions, billing records migrated
Storage	Cloud Storage	Cloud Storage	No change

Tenants	Est. Infrastructure Cost/mo	Est. Revenue	Gross Margin
200	~\$1,800 (hybrid)	~\$39,800	~95%
350	~\$2,800	~\$69,650	~96%
500	~\$3,500	~\$99,500	~96%

STAGE 3 — Full Cloud-Native PostgreSQL (500+ Tenants)

At this scale the platform has proven revenue to invest in proper infrastructure. Full PostgreSQL migration with dedicated analytics, multi-region support, and enterprise reliability. Firebase phased out except mobile SDK for offline sync if needed.

Layer	Service	Notes
Primary DB	PostgreSQL (AWS RDS or Supabase)	Full relational model, row-level security
Real-time	Supabase Realtime or Ably	Replaces Firestore listeners
Auth	Supabase Auth or Auth0	Adapter swap — zero business logic change
Storage	AWS S3 or Cloudflare R2	Lower cost than Firebase Storage at scale
Backend	Cloud Run or AWS Lambda	More flexibility, lower cold starts
Analytics	PostgreSQL + BigQuery	Real BI queries on full dataset

Tenants	Est. Infrastructure/mo	Est. Revenue (blended tiers)	Gross Margin
500	~\$3,000	~\$150,000+	~98%
1,000	~\$5,000	~\$300,000+	~98%
2,000	~\$8,000	~\$600,000+	~99%

27.4 Migration Risk Mitigation

Risk	Mitigation
------	------------

Data loss during migration	Dual-write pattern — data written to both old and new systems simultaneously during transition
Real-time breaks	Firestore listeners kept active until replacement validated — no cutover until feature parity confirmed
Auth tokens invalidated	Firebase Auth tokens honored for 30-day overlap after new auth system is live
Mobile app breaks	Mobile app uses adapters — backend URL and auth are config, not hardcoded
Developer skips abstractions	Code review gate: no direct SDK imports outside adapters/repositories before merge
Migration disrupts customers	All migrations use zero-downtime blue-green deployment — no customer interruption ever

27.5 Non-Negotiable Phase 1 Deliverables

These architectural deliverables must exist before the platform launches. They cost ~20% more time upfront and save 300%+ in future migration costs.

- /repositories directory — one Repository class per Firestore collection, no direct db calls anywhere else
- /adapters directory — one Adapter class per external service, no direct SDK imports anywhere else
- /config directory — single config file reading all environment variables
- README documenting the abstraction architecture and how to add a new repository or adapter
- Code review checklist item enforced on every PR: ‘No direct Firestore/Firebase SDK imports outside repositories and adapters’
- Unit tests for all repository and adapter classes — these are the most critical tests in the codebase

27.6 Technology Comparison

Criterion	Firebase (Stage 1)	PostgreSQL (Stage 2-3)	Best At Scale
Speed to build	Fastest	Fast	Firebase
Cost at 50 tenants	~\$600/mo	~\$200/mo	PostgreSQL
Cost at 500 tenants	~\$6,000+/mo	~\$1,500/mo	PostgreSQL
Real-time updates	Native, excellent	Very good	Firebase
Complex queries / analytics	Weak	Excellent (SQL)	PostgreSQL
Relational data integrity	Not supported	Native	PostgreSQL

Vendor lock-in	High	Low	PostgreSQL
DevOps required	None	Minimal	Firebase
Mobile offline support	Excellent	Good	Firebase
Migration path	Painful if not abstracted	Easy with adapters	PostgreSQL

Strategy: Build on Firebase with proper abstractions from day one. Migrate analytics and financial data to PostgreSQL at ~200 tenants. Complete migration at ~500 tenants. No full rewrite ever required if architecture is built correctly.

28. Glossary

Term	Definition
Tenant	A roofing company that subscribes to the Roof-Aid platform
Rufero	Field inspector/salesperson who visits properties and conducts roof inspections
Telefonista	Inside sales call agent who contacts leads by phone, SMS, and email
Prospect	A homeowner record in the system — a potential roofing customer
Lead	A prospect at the earliest pipeline stage, typically from storm data
ACV	Actual Cash Value — insurance payout at depreciated roof value
RCV	Replacement Cost Value — full insurance payout for roof replacement
Supplement	Additional insurance claim submitted after initial estimate to recover full costs
Hail Size	Diameter of hail in inches — determines damage severity
Scope of Damage	Detailed assessment of roof damage used for insurance claims
3rd Party Authorization	Document allowing roofer to represent homeowner with their insurance company
Proximity Search	Search for prospects within a defined radius of a map point (geospatial query)
Multi-tenant	Single platform serving multiple companies with complete data isolation
Feature Flag	Runtime switch that enables/disables a feature based on subscription tier
WebRTC	Web Real-Time Communication — browser-based phone calling technology
Extension	Personal Telnix phone line assigned to an individual agent
Geohash	String encoding of lat/lng coordinates — used for efficient proximity queries in Firestore

Cloud Function	Serverless backend function that runs on Google Cloud — scales automatically
Stripe	Payment processing platform — handles subscriptions and commission billing
SendGrid	Email delivery service — each agent has a personal sending subaccount
DNC	Do Not Call — legal requirement prohibiting outreach to flagged numbers
TCPA	Telephone Consumer Protection Act — US law governing calls and SMS to consumers
PWA	Progressive Web App — web app installable on any device like a native app
MRR	Monthly Recurring Revenue — total subscription revenue per month
Commission	10% of supplement claim value charged to tenant when supplement is processed
Provisioning	Automated setup of a new tenant account including phone, email, and database
Dunning	Automated process to recover failed subscription payments
CI/CD	Continuous Integration / Continuous Deployment — automated code testing and deployment
tenantId	The unique identifier for a roofing company — used as data partition key in all DB operations

29. Proprietary Modules & IP Licensing

Three modules incorporated into Roof-Aid CRM are proprietary technology developed prior to this project and are owned by the founding technical team. The developer building this system is working with licensed implementations of these modules — not building replacements from scratch.

29.1 Licensed Modules

Module 1: Motor Geospacial (Geospacial Engine)

Covers the proximity search, geohash-based radius query, and map pin clustering system. The core algorithm for converting lat/lng coordinates to geohashes, running radius queries in Firestore, and clustering results for map display is the licensed IP. The developer must integrate the existing implementation, not write a new geospacial query engine.

Module 2: Hub de Telecomunicaciones (Telecommunications Hub)

Covers the WebRTC softphone integration, Telnyx extension routing, personal caller ID management, inbound call routing by extension, and SMS threading per agent. The architecture of personal-extension-per-agent, the webhook routing logic, and the WebRTC session credential generation are the licensed IP.

Module 3: Sistema Anti-Colision (Anti-Collision System)

Covers the cyclic temporal rotation algorithm for prospect list distribution among concurrent telefonistas. See Section 30 for full technical specification. This module prevents multiple agents from calling the same prospect simultaneously without requiring database locks.

29.2 License Terms (Summary)

- The product owner receives a license of use for all three modules exclusively for the Roof-Aid CRM product in the US roofing contractors market
- The license does not permit sublicensing, reselling, or use in other products or markets without separate written agreement
- The backend Cloud Functions implementing these modules remain on infrastructure controlled by the technical co-founder during the initial development phase
- Frontend code is shared via Git repository; backend Functions are not included in the Git repo and are transferred separately only upon explicit agreement
- If the partnership ends, the backend Functions may be purchased at a separately negotiated price with a 2-5 day technical transfer window

29.3 Developer Instructions

When implementing any of the three modules above:

1. Request the existing implementation from the technical co-founder before writing any replacement code
2. Wrap the implementation in the appropriate Adapter or Repository class per the architecture pattern (Section 27)
3. Do not reverse-engineer or rewrite the core algorithms — integrate the provided code
4. If the provided implementation needs modification, discuss with the IP owner before changing
5. Anti-Collision System (Sistema Anti-Colision)

30.1 The Problem

When multiple telefonistas work simultaneously on the same pool of prospects, without coordination they will naturally call the same prospects at the same time. This wastes agent time, creates a poor homeowner experience (receiving two calls from the same company), and makes disposition tracking unreliable. Database locks are too expensive and slow at Firestore's pricing model.

30.2 The Solution: Cyclic Temporal Rotation

The system loads 60 prospects (not the intuitive 50) per page. The display order of those 60 records is rotated based on the current second of the wall clock at the time the agent loads the list (0-59 seconds).

Each agent effectively sees the same 60 prospects but in a different order based on when they loaded. The prospect at position #1 for Agent A is at a different position for Agent B unless they loaded at exactly the same second.

30.3 Mathematical Basis

- Probability two agents load at the exact same second: $1/60 = 1.67\%$
- Probability three agents all load at the same second: $(1/60)^2 = 0.028\%$
- Probability four agents all load at the same second: $(1/60)^3 = 0.00046\%$

For K concurrent telefonistas to all prioritize the same prospect, they would need to have loaded their list at the exact same second — an effectively negligible probability in real-world usage.

30.4 Implementation Requirements

```
function loadProspects(tenantId, status, city) {  
  // Load 60 records (not 50) from Firestore, ordered by priority  
  index  
  const raw60 = await ProspectRepository.getPage(tenantId, {  
    status, city, limit: 60  
  });  
  
  // Get current second as rotation offset (0-59)  
  const offset = new Date().getSeconds();  
  
  // Rotate: move first 'offset' items to end  
  const rotated = [...raw60.slice(offset), ...raw60.slice(0, offset)];  
  
  // Display all 60 in rotated order  
  return rotated;  
}
```

30.5 Interaction with Status State Machine

When an agent marks a prospect as “Contacted” or changes its status:

1. The prospect immediately disappears from the agent’s own list (client-side filter)
2. The Firestore document is updated with the new status and a lastContactedAt timestamp
3. The next time any other agent loads or refreshes their list, that prospect no longer appears (it no longer matches the status=Prospects filter)
4. No database lock is required — the status update itself serves as the coordination mechanism

30.6 The “Bring Next” Pattern

When a prospect is removed from the displayed list (status change, DNC, etc.), the system should automatically fetch the next record from Firestore to maintain 60 visible records. This keeps agents continuously working without manual refreshes.

```
function onProspectStatusChanged(prospectId, newStatus) {  
  // Remove from local display list  
  removeFromList(prospectId);  
  
  // Fetch replacement (cursor after last loaded record)  
  const next = await ProspectRepository.getNextAfterCursor(tenantId,  
    cursor);  
  if (next) appendToList(next);  
}
```

30.7 Why 60 and Not Another Number

The number 60 was chosen because the rotation offset is the wall clock second (0-59), giving exactly 60 discrete starting positions. Using 50 records would mean rotation offsets 50-59 would wrap around and duplicate the top of the list, defeating the purpose. The number must equal or exceed 60.

31. Existing Production System & v1 to v2 Migration

31.1 Context: What Already Exists

Roof-Aid CRM is not a greenfield project. A production system (v1) has been running for over 3 months with 4-5 active telefonistas. It contains:

- ~30,000 real homeowner prospect records across multiple geographic markets
- Live call logs, SMS threads, appointment records, and signed documents
- A working Telnyx WebRTC softphone integration
- An active single-tenant Firestore database with 18 composite indexes
- A Vanilla JS frontend (25+ modules using window.* globals) with Vite bundler
- Firebase Auth with the 4 existing roles (owner, admin, telefonista, rufero)

The v2 system described in this spec is a React multi-tenant rebuild of v1. It must be built, tested, and launched without disrupting v1 for existing users.

31.2 Architecture of the Existing v1 System

The v1 frontend uses a window globals pattern (25+ variables exposed on window.*) with direct DOM manipulation and no client-side router. There is no React or reactive state. The 13 Cloud Functions in v1 map as follows to the v2 architecture:

- 8 callable functions: WebRTC auth, email, SMS, assignment logic, workflow, PDF generation, calendar, analytics
- 3 HTTP endpoints: Telnyx webhooks, SMS delivery status webhooks, SendGrid inbound parse
- 2 scheduled functions: Daily metrics (3am), garbage collection (2am)

All 13 must have functional equivalents in v2 before v1 can be decommissioned.

31.3 Known v1 Limitations Being Fixed in v2

- Single-tenant: all clients share one Firestore project (no tenantId partition)
- Queries load entire collections (e.g., all 30k prospects) without per-tenant filtering
- Email uses one shared sender domain for all agents
- Phone uses one shared number without personal extensions
- No feature flags: all features available to all users
- No automated onboarding: new client setup takes 4-8 hours manually

31.4 Zero-Downtime Migration Strategy

Use a blue-green deployment approach:

- Phase A (Build): v2 is built and tested on a separate Firebase project (staging). v1 continues serving all users. No changes to v1.
- Phase B (Pilot): Select 1-2 clients as v2 pilot tenants. Migrate their data to the v2 multi-tenant schema. They use v2 while others continue on v1. Run both systems simultaneously.
- Phase C (Migration): Migrate remaining clients to v2 one at a time. v1 set to read-only mode per client as they move.
- Phase D (Decommission): Once all clients are on v2 and stable for 30 days, v1 Firebase project is decommissioned and archived.

31.5 Data Migration Script Requirements

A migration script must be written for each v1 client that:

1. Reads all records from the v1 single-tenant Firestore collections
2. Adds tenantId field to every record
3. Writes to the v2 multi-tenant Firestore with proper composite indexes
4. Validates record counts match before and after (zero tolerance for data loss)
5. Runs in under 8 minutes for a typical client dataset (~30k prospects + related records)
6. Is idempotent (can be re-run safely if interrupted)
7. Produces a migration report: records migrated, duplicates skipped, errors

31.6 The 18 Existing Firestore Indexes

The v1 system has 18 composite indexes that are production-tested. The v2 firestore.indexes.json must include all 18 of these PLUS new indexes for the tenantId prefix on each. The 5 v1 indexes not in the original v2 spec are:

- prospects: geohash + tenantId + status (anti-collision proximity queries)
- prospects: city + homeValue + createdAt (priority sorting by home value)
- call_logs: agentId + tenantId + createdAt (per-agent call history)
- user_sessions: tenantId + userId + createdAt (session management audit)
- activities: tenantId + type + createdAt (platform-wide activity analytics)

31.7 Rollback Plan

If v2 has critical issues after a client is migrated:

- v1 is kept running in read-only mode until v2 stability is confirmed
- Rollback time target: under 15 minutes (DNS switch back + read-only disable)
- All v2 writes are also written to a migration_log collection for recovery
- Client data is never deleted from v1 until the client has been stable on v2 for 30 days

END OF DOCUMENT

Roof-Aid CRM | Complete Developer & Business Specification v3.0 | February 2026 |

CONFIDENTIAL